

The Equalis Protocol

Matthew L. Hooft  

EqualFi Labs

E-mail: mhooft@equalfilabs.com (Matthew L. Hooft)

This work is licensed under a “[CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)” license.
Date of this document: 2026-02-14.



Abstract. Decentralized finance has largely converged on a reactive pattern: liquidation-based lending, oracle-driven solvency checks, and fragmented venues that silo liquidity and incentives. These systems can function, but they impose continuous operational burden on users, introduce forced-selling failure modes during volatility, and expand discretionary control surfaces (oracle selection, liquidation pathways, and governance knobs).

Equalis proposes an alternative foundation built around determinism and user agency. The protocol provides **0% interest self-secured credit** (same-asset borrowing against one's own deposits) and an explicit peer-to-peer credit layer for cross-asset borrowing. Solvency in the deterministic core does not depend on external price feeds or liquidation auctions. When cross-asset risk is desired, it is expressed through explicit credit agreements with transparent terms, optional covenants (including an opt-in lender call feature), and default handling that treats defaults as defaults rather than liquidation opportunities.

Equalis Direct also supports **P2P options** (synthetic, physically settled option payoffs) expressed as collateralized agreements where the borrower's repay-versus-forfeit decision maps to exercise. Strikes are encoded by fixed collateral-to-principal ratios rather than oracle-priced cash settlement. These agreements use the same event-indexable offer flow as peer-to-peer credit, and option portfolios can be packaged and transferred by transferring the Position NFT that contains the agreements.

Equalis also targets liquidity fragmentation directly. The system enforces a **canonical base pool per token** and uses an **encumbrance** model: capital allocated to internal venues is marked as encumbered via accounting rather than leaving the pool. This enables shared liquidity and fee routing across protocol surfaces, while keeping accounting coherent and reducing multi-contract asset hopping.

Value is returned to participants through a **dual index system**: a *Fee Index* that rewards passive liquidity providers and an *Active Credit Index* that rewards deployed, productive capital (borrowed principal, underwriting, and encumbered module inventory). Protocol modules can pay direct counterparties (for example market makers) while also routing a share of fees into the same per-token indexes.

A central fee-generating module is **EqualIndex**, a deterministic, fully backed basket token primitive. Each index defines a fixed bundle of underlying assets at creation, with no rebalancing and no external pricing oracles. EqualIndex supports mint, burn, and **basket flash loans** that borrow proportional amounts of each underlying asset in a single transaction. These flash loans are intended as a deterministic arbitrage and execution device for multi-leg routes. Fees collected per underlying asset are split between: (i) a pool share routed through the system fee router (Treasury, Active Credit Index, and the underlying pool Fee Index) and (ii) a fee pot that accumulates for index holders and is realized proportionally on redemption.

Equalis also includes MEV-aware execution venues. **Maker Auction Market (MAM) curves** provide linear time-based pricing that can be descending, ascending, or flat (limit-order-like), and can approximate iceberg-style liquidity through staged curves. Because MAM pricing is determined by time rather than trade-induced reserve shifts, it reduces common slippage-based MEV vectors seen in reserve-based AMMs. Curves are cheap and batchable to update, enabling higher-frequency maker adjustment without requiring new pools or new venues.

This paper defines the architecture for unified pools and encumbrance, the index mechanics and fee routing, the EqualIndex basket model and fee pots, MAM curve execution, and a precise notion of *protocol-perpetual leverage*: leverage loops that are not subject to price-triggered forced unwinds, persisting until the borrower chooses to unwind or fails explicit contractual obligations.

Key Claims (and Non-Claims)

- **Claim: Deterministic core credit.** Self-secured credit does not require oracles or liquidation markets to maintain solvency.
- **Claim: Unified liquidity by token.** For each token, there is a canonical base pool that aggregates liquidity and fee routing across protocol surfaces, with optional managed pools that route a system share back to the base layer.
- **Claim: Dual-index incentives.** Passive liquidity and active deployment are rewarded through separate indexes to keep incentives legible.
- **Claim: Deterministic basket liquidity.** EqualIndex provides fixed-composition, fully backed basket tokens and proportional basket flash loans without external pricing oracles.
- **Claim: MEV-aware execution surfaces.** MAM curves enable time-based pricing (including limit-order-like flat curves) and reduce trade-induced repricing MEV vectors common to reserve-based AMMs.
- **Claim: P2P options without cash-settlement oracles.** Synthetic options can be expressed as physically settled Direct agreements with strikes encoded by fixed ratios.
- **Claim: System-wide fee routing.** Module fees can compensate direct participants while also routing value into the same per-token Fee Index and Active Credit Index.
- **Claim: Protocol-perpetual leverage.** The protocol contains no rule that forces collateral sale solely due to mark-to-market price changes; continuation is governed by explicit obligations (payments and agreed covenants).
- **Non-claim: Guaranteed yield or profitability.** Index returns depend on realized protocol activity and fees.
- **Non-claim: Risk elimination.** Peer-to-peer credit includes counterparty default risk; smart contract and module risks exist and must be addressed by engineering and isolation.
- **Non-claim: Price-independence of feasibility.** Market liquidity and execution conditions may affect whether strategies are attractive or feasible to initiate, but do not introduce price-triggered forced unwinds in the deterministic core.
- **Non-claim: MEV elimination.** MAM curves reduce certain MEV classes (especially slippage-based sandwiching) but cannot remove ordering competition in a public mempool.

1 Problem Statement

Onchain finance has made major progress, but much of modern DeFi has converged on a set of assumptions that are treated as inevitable:

1. *Credit must be liquidation-based to be safe.*
2. *Solvency must be oracle-driven.*
3. *Risk containment requires separate venues and siloed liquidity.*

These assumptions are not irrational; they are pragmatic responses to building under adversarial conditions. However, they also introduce structural failure modes and user costs that are now widely normalized.

1.1 Reactive Systems and the Cost of Vigilance

Liquidation-based lending is fundamentally reactive. Borrowers are required to continuously maintain a *health factor* relative to an externally measured price. When market volatility occurs, risk is often resolved by forced selling and closing positions at the worst moment turning transient drawdowns into realized losses. This produces a user experience where leverage is not only risky, but operationally demanding:

- borrowers must monitor positions continuously,
- positions can be liquidated through no fault of the user beyond market movement,
- liquidation mechanisms incentivize third-party extraction during stress events.

From a system perspective, reactive solvency creates correlated behavior. Many users face liquidation pressure at the same time, which can amplify volatility and concentrate profit in liquidation pathways rather than in productive use of capital.

1.2 Oracle Dependency as a Control Surface

Oracle reliance is often treated as an engineering detail, but it is a major trust and capture surface. Any system that requires an external price feed to determine solvency inherits:

- *feed selection risk* (which oracle, which market, which aggregation rules),
- *update and latency risk* (stale or delayed information under stress),
- *manipulation and coordination risk* (thin markets, outliers, or adversarial routing),
- *governance risk* (who can change oracle parameters and under what conditions).

Even when oracles perform well, their existence changes the nature of the protocol. The system is no longer defined purely by onchain invariants; it is partially defined by external measurement and the processes that maintain it.

1.3 Liquidity Fragmentation and Incentive Islands

Most DeFi stacks are composed of multiple specialized venues (lending, AMMs, perps, options, indices, prediction markets). In the typical architecture, capital moved to one venue becomes idle with respect to others. This creates two recurring inefficiencies:

1. **Fragmented liquidity:** each venue must bootstrap and retain its own liquidity, often leading to shallow markets and duplicated incentives.
2. **Incentive hopping:** users chase the highest short-term rewards, which encourages emissions-driven growth and reduces stability of long-term liquidity.

The result is a system where capital is frequently underutilized and where protocol growth often depends on temporary subsidies rather than durable fee-based value flow.

1.4 Design Goals

Equalis is designed to challenge the above assumptions by shifting the core of onchain credit from reactive liquidation to deterministic accounting and explicit agreements. The design goals are:

- **Deterministic solvency for the core:** enable credit that does not require price feeds or liquidation markets to remain solvent.

- **User agency over forced selling:** replace health-factor monitoring with explicit terms and obligations.
- **Unified liquidity per token:** maintain a canonical base pool per asset and route fees from multiple protocol surfaces back into the same accounting base (with optional managed pools described below).
- **Legible incentives without emissions reliance:** reward passive liquidity and active deployment separately using a dual-index model.
- **Walkaway-orientation:** minimize discretionary control surfaces and define a path toward progressively immutable protocol behavior.

1.5 What This Paper Does (and Does Not) Attempt

This paper describes an architecture for deterministic credit primitives, unified liquidity accounting via encumbrance, and shared fee routing through dual indexes. It does not claim to eliminate risk or guarantee profitability. Instead, it aims to make risk explicit, reduce forced unwinds as a default enforcement mechanism, and improve system-wide capital utilization by preventing liquidity and incentives from fragmenting across protocol surfaces.

2 System Overview

Equalis is a modular onchain financial system built around a deterministic core and unified per-token liquidity. The system is defined by four structural choices:

1. **One canonical pool per token** (unified liquidity and unified fee routing, with optional managed pools),
2. **A Position NFT** that represents a complete onchain balance sheet,
3. **Encumbrance-based allocation** that enables internal venues without fragmenting liquidity,
4. **A shared fee router and dual indexes** that distribute value to passive and active participants.

On top of this foundation, Equalis supports opt-in modules that generate fees (and therefore index value), including markets and credit agreements, AMM auction venues, and **EqualIndex** (a deterministic, fully backed basket and basket flash loan primitive).

2.1 High-Level Architecture

For each supported token T , Equalis maintains a canonical base pool $\mathcal{P}_T$. All protocol surfaces that use token T reference $\mathcal{P}_T$ as the canonical liquidity source and the canonical destination for fee routing in token T .

Managed pools (implementation). The current implementation also supports **managed pools**: additional pools for the same underlying asset with custom parameters, optional whitelists, and independent accounting. Managed pools route a configurable system share of fees back to the canonical base pool (or treasury when no base pool is active). This preserves unified fee rails at the base layer while enabling curated or permissioned pool variants.

Users interact with the system through **Position NFTs**. A Position NFT holds a complete account state: deposits, liabilities, module allocations, and accrued claims on per-token indexes. Protocol modules allocate inventory via encumbrance accounting rather than extracting liquidity into isolated sub-pools. Fees produced by modules route back into the same per-token accounting rails.

2.2 Core Components

2.2.1 Global Token Pools

Each token T has a canonical base pool $\mathcal{P}_T$. This pool is the shared accounting and liquidity substrate for all actions involving T . The base pool tracks:

- total deposited liquidity,
- total encumbered liquidity (allocated to modules),
- pool parameters governing deterministic credit bounds (e.g., self-secured LTV),
- per-token fee and reward accounting via indexes.

Pool membership and gating (implementation). Pool interactions are membership-gated in the current codebase. Membership can be auto-joined on first interaction, but managed pools can enforce explicit whitelist membership. Membership can only be cleared when balances and obligations are settled.

2.2.2 Position NFTs

A Position NFT is an all-in-one onchain balance sheet. It stores the state required to compute:

- internal ledger balances per token pool,
- same-asset debt (self-secured credit),
- collateral locked under explicit peer-to-peer agreements,
- encumbered balances allocated to protocol modules,
- accrued claims on the Fee Index and Active Credit Index for each relevant token.

Transferring the NFT transfers the position (assets, obligations, and index claims), subject to transfer rules needed to preserve solvency and module invariants.

2.2.3 Encumbrance Accounting

Encumbrance is the allocation mechanism that allows capital to participate in internal protocol venues without leaving the global pool. When a position allocates inventory to a module, the protocol:

1. marks the allocated amount as encumbered on the position,
2. records module-specific encumbrance state (e.g., “auction reserves”, “direct-locked collateral”, “index backing”),
3. prevents double-use of the same units of liquidity,
4. performs internal ledger moves where possible to reduce external transfers and preserve unified liquidity.

Encumbrance is not a promise of risk-free reuse. It is a strict accounting mechanism that makes capital *available to modules* while making it *unavailable for conflicting uses*.

2.2.4 Fee Router and Dual Indexes

All fees collected in token T are routed through the fee router associated with $\mathcal{P}_T$. Fee routing can produce:

- direct payouts to module participants (e.g., makers in AMM auctions),
- protocol revenue (e.g., treasury allocation),
- a contribution to the **Fee Index** for token T (passive leg),
- a contribution to the **Active Credit Index** for token T (active leg).

Each pool maintains two monotonic indexes:

$$\begin{aligned} I_T &:= \text{Fee Index for token } T, \\ A_T &:= \text{Active Credit Index for token } T. \end{aligned}$$

Positions accrue claims on I_T and A_T according to earning weights defined by protocol rules (specified in later sections). The design intent is to reward passive liquidity without starving the capital that is actively deployed into credit and markets.

2.3 Modules as Fee-Producing Surfaces

Equalis supports opt-in modules that expand what pooled capital can do. Modules are *surfaces* that generate fees and settle deterministically, not separate liquidity systems. In particular:

2.3.1 Deterministic Credit and Explicit Credit Agreements

- **Self-Secured Credit:** same-asset borrowing against one's own deposits at 0% interest within deterministic bounds.
- **Equalis Direct (P2P):** explicit lending agreements for cross-asset borrowing with transparent terms, optional covenants, and deterministic default handling.

2.3.2 AMM Auction Venues

- **Solo AMM Auctions:** a single position can create a time-bounded AMM-style auction using encumbered reserves.
- **Community Auctions:** multiple positions contribute liquidity to a shared auction, with maker fee distribution.

In both cases, fees can compensate makers while routing a configured share into the per-token fee rails.

2.3.3 EqualIndex: Deterministic Basket Tokens and Basket Flash Loans

EqualIndex is a permissionless index token primitive for creating static, fully backed basket tokens. An index is defined once at creation with fixed composition and no rebalancing. Each index unit represents a deterministic bundle of underlying assets.

EqualIndex provides:

- **Mint and burn:** minting deposits the exact bundle; burning redeems a proportional share of backing.

- **Fee pots:** each underlying asset maintains an index-specific fee pot that accumulates fees for index holders and is realized on redemption.
- **Basket flash loans:** a borrower can atomically borrow proportional amounts of each underlying asset for a chosen number of index units, use the inventory for multi-leg execution (e.g., arbitrage), and return the full basket plus fees in the same transaction.

Fees collected per underlying asset are split between (i) an *index-holder fee pot* and (ii) a *pool share* routed through the system fee router (Treasury, Active Credit Index, and the underlying pool Fee Index). This makes EqualIndex a fee-producing execution primitive that strengthens unified per-token fee rails rather than creating an isolated incentive island.

2.3.4 Pool Flash Loans (Implementation Detail)

In addition to basket flash loans, the base pools themselves expose single-asset flash loans with deterministic fee routing and optional anti-split protection. These are fee-producing surfaces and integrate with the same fee router, but they do not rely on external pricing.

2.4 Key System Properties

The architecture aims to preserve the following properties:

- **Unified liquidity:** a canonical base pool per token across the entire system (with optional managed pools).
- **Coherent accounting:** encumbrance prevents double-use and keeps obligations legible.
- **Composability without fragmentation:** modules can be added as fee-producing surfaces while sharing the same per-token fee rails.
- **Legible incentives:** dual indexes separate rewards for passive liquidity and active deployment.
- **Reduced discretion:** deterministic core behavior minimizes reliance on price feeds and liquidation markets.

The remainder of this paper specifies the deterministic credit primitives, the encumbrance model, index mechanics, EqualIndex fee routing and basket flash loans, and the implications for capital efficiency and leverage strategies.

3 Deterministic Core Credit

Equalis begins with a deterministic credit primitive designed to remove the most common operational failure mode in DeFi lending: price-triggered forced unwinds. The core credit layer is built around **self-secured borrowing** (same-asset credit against one's own deposits) and is complemented by an explicit peer-to-peer credit system for cross-asset borrowing. This section focuses on the deterministic core.

3.1 Self-Secured Credit (0% Interest)

3.1.1 Definition

For a token T with pool $\mathcal{P}_T$, a position may borrow token T against its own deposited balance of T up to a fixed loan-to-value (LTV) bound:

$$D_{p,T} \leq \lambda_T \cdot \text{Deposit}_{p,T},$$

where:

- $D_{p,T}$ is the position's same-asset debt in token T ,
- $\lambda_T \in (0, 1)$ is a pool parameter (e.g., $\lambda_T = 0.95$),
- $\text{Deposit}_{p,T}$ is the position's deposited principal of T .

Self-secured credit is **0% interest**. The borrower does not pay an interest rate that compounds over time. Instead, the protocol's economic model is based on fee-generating activity across protocol surfaces that route value into shared indexes (described in later sections).

Interest-free does not mean fee-free (implementation). The current implementation charges explicit *action fees* (borrow/repay/withdraw/flash/close), applies a pool-level *maintenance fee* (accrued via a negative index on deposits), and enforces minimum payment rules for rolling loans. These are deterministic and onchain, and they are not price-dependent.

3.1.2 Why Solvency Does Not Require Oracles

In self-secured credit, collateral and debt are denominated in the same token. Solvency is therefore not a function of external price movements. No mark-to-market conversion is required to decide whether the position remains solvent:

- the debt is a claim on T ,
- the collateral is held in T ,
- the protocol enforces the bound $D_{p,T} \leq \lambda_T \cdot \text{Deposit}_{p,T}$ deterministically.

This design removes the need for external price feeds and liquidation auctions to preserve solvency in the deterministic core. The protocol does not need to decide what T is “worth” relative to other assets to keep the position solvent in T terms.

3.1.3 User Experience and Agency

Because solvency does not depend on a price oracle or health factor, the deterministic core does not require continuous user vigilance. A user can choose to:

- remain purely passive (deposit only),
- borrow against themselves to unlock liquidity,
- deploy borrowed liquidity elsewhere (inside or outside the protocol),
- allocate deposited liquidity to internal fee-producing modules.

The position is not force-closed solely due to price volatility.

3.2 Active Credit as a First-Class Behavior

Equalis treats deployed capital as valuable rather than suspicious. Borrowed principal and other forms of active deployment expand what the system can do (credit, market making, execution, arbitrage), so the protocol rewards active participation through a distinct accounting rail: the **Active Credit Index**.

In the deterministic core, self-secured borrowing is therefore not merely “allowed” it is designed to be *economically legible*. Borrowing increases activity, and activity contributes to fee generation across modules that can route value back to active participants.

3.3 Fixed-Term Self-Secured Loans (Implementation Detail)

The current implementation also supports fixed-term self-secured loans alongside rolling credit. These fixed-term loans are still same-asset and oracle-free; expiry is handled via deterministic repayment and penalty rules rather than price-based liquidation.

3.4 Preventing Trivial Loop Farming (High-Level)

A system that rewards activity must also ensure that activity is costly to fake. Equalis uses explicit accounting rules to prevent trivial “borrow and deposit” loops from extracting rewards without producing system value. At a high level, the protocol separates:

- **passive earning weight** (Fee Index) from
- **active earning weight** (Active Credit Index),

and applies constraints to ensure the same units of capital are not counted multiple times toward the same reward stream without providing corresponding fee-generating behavior.

Formal earning weights and anti-gaming rules are specified in the dual index section. Here, the key point is architectural: **self-secured credit can be 0% interest and high-LTV because it is self-denominated, while incentives are controlled through index accounting rather than interest rate spreads.**

3.5 Protocol-Perpetual Continuity in the Deterministic Core

The deterministic core is designed so that positions are not subject to protocol-enforced forced unwinds triggered by mark-to-market price changes. In the self-secured credit primitive:

- there is no oracle threshold that triggers collateral sale,
- there is no liquidation auction that closes the position due solely to volatility,
- continuation is governed by deterministic bounds in the same asset denomination.

This property is foundational for later strategies discussed in this paper, including leverage loops that depend on continuity through drawdowns without relying on liquidation market behavior.

Delinquency enforcement (implementation). While there are no oracle-triggered liquidations, the current implementation does enforce delinquency rules: missed payment thresholds or expired fixed-term loans can trigger deterministic penalty and collateral seizure flows. These are obligation-triggered (not price-triggered) and are part of the deterministic enforcement surface.

3.6 Relationship to Explicit Credit (Preview)

When a user seeks to borrow an asset they do not already own, Equalis moves from deterministic credit to explicit credit agreements. In that domain, risk is not eliminated; it is made legible through contract terms, optional covenants (including an opt-in lender call feature), and deterministic default handling. The peer-to-peer credit system is specified in the next section.

3.7 Equalis Direct: Explicit Peer-to-Peer Credit

Self-secured credit is intentionally limited to same-asset borrowing. When a borrower wants to obtain an asset they do not already own, the protocol shifts from deterministic solvency to **explicit agreements**. Equalis Direct is a peer-to-peer lending system where credit risk is not hidden inside pooled parameters; it is expressed directly in contract terms.

3.7.1 Design Principle: Risk Must Be Legible

In many pooled lending systems, a borrower interacts with a shared pool and a protocol-defined risk model. This model often relies on oracle pricing and liquidation markets, and it can hide important assumptions behind governance-controlled parameters.

Equalis Direct instead makes the deal explicit:

- **Borrowers** choose terms they are willing to accept.
- **Lenders** choose risk they are willing to underwrite.
- **The protocol** enforces the agreement deterministically, without discretionary intervention.

3.7.2 Agreement Structure

A peer-to-peer agreement is defined by parameters such as:

- loan asset and principal amount,
- collateral asset and locked collateral amount,
- repayment schedule (e.g., monthly payment),
- term structure (term loan or rolling structure),
- grace windows and default conditions,
- fee or pricing terms (e.g., upfront interest, periodic payments),
- optional covenants (including an opt-in lender call feature).

The protocol treats these parameters as the ground truth. There is no requirement that solvency be continuously evaluated using external price feeds. If a lender wants additional protection, it is expressed in the agreement terms (collateral ratio, callability, stricter grace windows), not delegated to liquidation markets.

3.7.3 Term and Rolling Credit

Equalis Direct supports two agreement shapes for cross-asset credit:

- **Term credit:** agreements with a fixed due timestamp and explicit grace/default handling.
- **Rolling credit:** agreements with a recurring payment interval and a moving due timestamp (`nextDue`). A rolling agreement is kept current by making payments that clear accrued arrears and the current interval interest; `nextDue` advances by one interval only when the agreement is brought current. If `block.timestamp` exceeds `nextDue` plus a grace window, recovery becomes available.

This is a structural distinction from liquidation-based systems. Continuation is governed by contractual compliance rather than by maintaining a health factor against a price oracle.

3.7.4 Callable Loans as an Opt-In Covenant

Equalis Direct includes an **optional lender call** feature. This is not a universal lender power; it is a covenant that must be accepted as part of the agreement.

Let `allowLenderCall` $\in \{0, 1\}$ be a flag set on an agreement:

- If `allowLenderCall` = 0, the lender cannot unilaterally accelerate repayment.
- If `allowLenderCall` = 1, the lender may invoke a call that accelerates the agreement according to protocol rules (e.g., setting the due timestamp to the current time and starting the grace window immediately).

Borrowers choose whether to accept callable terms by selecting which offers to fill. This preserves borrower agency while allowing lenders to offer differentiated products for differentiated risk appetites.

3.7.5 Default Handling: Defaults Are Not Liquidations

In Equalis Direct, a missed obligation is treated as a default event under the agreement rules. The protocol does not transform default into an adversarial liquidation race designed to maximize third-party extraction. Instead:

- default conditions are defined in the agreement (e.g., missed payment beyond grace),
- consequences are deterministic (e.g., collateral seizure, penalties, and fee routing),
- enforcement does not require an oracle-triggered auction mechanism.

The intent is not to claim defaults are harmless. The intent is to ensure enforcement is legible and incentive-compatible, without importing liquidation MEV as a core design requirement.

Grace windows and default splits (implementation). The current implementation applies fixed grace windows around repayment/exercise, and default recovery routes collateral through deterministic splits: a lender share plus a protocol/index share routed via the fee router. These parameters are explicit and bounded.

3.7.6 Economic Integration with the Dual Index System

Equalis Direct is integrated into the system-wide incentive rails:

- capital locked in agreements as well as capital lent can be considered active deployment and earns exposure to the Active Credit Index according to protocol-defined rules,
- fees and penalties produced by agreements can be routed through the shared fee router into the per-token Fee Index and Active Credit Index,
- the underlying pools remain unified; agreements allocate capital via encumbrance rather than fragmenting liquidity into separate venues.

This means peer-to-peer credit is not an isolated product. It is a fee-producing surface that strengthens the unified per-token pools and the dual index accounting model.

3.7.7 Onchain Offer Book and Event-Indexable Order Flow

Equalis Direct is designed so that **both lenders and borrowers can post offers onchain**. Rather than requiring offchain negotiation, the protocol supports an offer-driven workflow that resembles an onchain credit order book:

- **Lender offers:** a lender posts standardized credit terms (principal, collateral requirements, payment schedule, covenant flags such as callability).
- **Borrower offers:** a borrower posts desired terms (requested principal, offered collateral, acceptable schedule and covenants).
- **Fills:** any counterparty can accept an offer that matches their preferences, causing the protocol to instantiate a binding agreement.

Offers and fills are emitted as structured onchain events. This makes the offer flow **indexable by external infrastructure** (indexers, explorers, analytics, or dedicated UIs) without requiring privileged coordination. In effect, the protocol can support a CLOB-like experience for credit discovery and matching, while keeping enforcement and settlement onchain and deterministic.

This design preserves user agency while enabling market-driven price discovery for credit. Standardized offers encourage liquidity and competition, while event indexing enables transparent visibility into outstanding terms, fill history, and evolving market conditions for onchain credit.

3.7.8 Tranche Offers: Term Tranches and Ratio Tranches

To make peer-to-peer credit scalable and liquid, Equalis Direct supports offer formats designed for repeated fills and market-style price discovery. Offers can be indexed by offchain infrastructure via events, enabling an onchain credit order book experience.

Offer Sides.

- **Lenders** can post *term tranche offers* and *ratio tranche offers*.
- **Borrowers** can post *ratio tranche offers* (borrowers do not post term tranche offers).

Term Tranche Offers (Lender-Posted, Tranche-Backed Term Credit). A *term tranche offer* escrows a fixed amount of principal at offer creation and allows that escrow to be filled across multiple counterparties over time. Conceptually, it behaves like a reusable liquidity slice for term credit.

Let an offer escrow **trancheAmount** of principal into the offer at post time. The protocol tracks:

$$\text{trancheRemaining} \leftarrow \text{trancheAmount} - \sum_i \text{fillPrincipal}_i.$$

Each fill creates a new bilateral agreement under the offer's terms, and decrements **trancheRemaining** until exhausted. This enables lenders to post a single standardized offer that supports many independent fills without re-posting or re-authorizing capital for each agreement.

Ratio Tranche Offers (Bid/Ask Credit Levels, Variable Fill Sizes). A *ratio tranche offer* generalizes offer flow into a CLOB-like format where principal can be drawn in variable sizes and collateral is determined by a posted ratio rather than by a single fixed fill size.

A ratio tranche offer defines:

- a principal capacity `principalCap` (for lender-posted offers, this is escrowed at post time),
- a minimum fill size `minPrincipalPerFill` to prevent dust,
- a deterministic ratio (n, d) with $n = \text{ratioNumerator}$ and $d = \text{ratioDenominator}$.

A fill selects any amount `principalFill` such that:

$$\text{minPrincipalPerFill} \leq \text{principalFill} \leq \text{principalRemaining}.$$

Collateral and principal requirements are derived deterministically from the ratio, depending on the offer side:

- **Lender-posted ratio offer (“ask” for credit):** the lender supplies `principalFill` and the borrower must provide collateral

$$\text{collateralRequired} = \text{principalFill} \times \frac{n}{d}.$$

- **Borrower-posted ratio offer (“bid” for credit):** the borrower specifies acceptable collateralization via the same ratio and requests `principalFill`; the borrower provides collateral according to

$$\text{collateralPosted} = \text{principalFill} \times \frac{n}{d},$$

and any lender may fill by supplying `principalFill` under the posted terms.

In both cases, each fill instantiates a concrete onchain agreement with explicit terms and deterministic enforcement, and decrements `principalRemaining` until exhausted.

Why Tranches Matter. Tranche offers turn peer-to-peer lending from one-off negotiation into a composable market primitive:

- lenders can deploy capital once and service many fills under standardized terms,
- borrowers can express demand directly by posting ratio offers at desired collateralization levels,
- indexers can present outstanding offers, fills, and ratio history as a transparent onchain credit market.

3.8 P2P Options via Equalis Direct (Synthetic Options)

Equalis Direct can express option-like payoffs *without* dedicated option contracts and *without* price oracles by using a collateralized lending agreement where the borrower’s terminal choice maps to the exercise decision.

This section describes these instruments as **synthetic options**: the payoff is option-like, but the mechanism is a Direct agreement with explicit terms and deterministic enforcement.

3.8.1 Core Insight: Repay vs Exercise Is the Option Decision

A Direct agreement ends with a borrower choice:

- **Repay:** return the borrowed asset and unlock collateral.
- **Exercise (non-recourse default):** keep the borrowed asset and forfeit locked collateral to the lender.

Economically, this is the same decision a rational option holder makes at expiry: exercise only when favorable relative to strike.

3.8.2 No Oracle Strike: The Strike Is Implied by the Ratio

A synthetic option agreement defines:

- `principal` (borrowed amount),
- `collateralLockAmount` (locked collateral),
- `durationSeconds` (time to expiry),
- premium terms (e.g., APR or upfront interest),
- exercise/repayment flexibility flags.

The **implied strike** is not computed on-chain. It is encoded by fixed quantities. For an agreement borrowing asset B against collateral asset C :

$$K_{\text{implied}} \equiv \frac{\text{collateralLockAmount}}{\text{principal}}.$$

This is a deliberate design choice: no onchain LTV checks, no price feeds, and no oracle manipulation vector. The lender chooses collateral requirements explicitly, and the market decides whether those terms clear.

3.8.3 Premium: Upfront Interest as the Option Premium

Synthetic options use **interest paid at origination** as the premium. At acceptance, the protocol can:

- lock borrower collateral via encumbrance,
- deduct premium from principal (or collect it explicitly),
- transfer net principal to the borrower,
- instantiate an active agreement record.

This is not “free optionality.” The lender is compensated up front for writing non-recourse exposure, and the borrower pays up front for the right to choose repay vs exercise.

3.8.4 Exercise Style: American vs European

Exercise style is expressed as an explicit term:

- **American-style:** `allowEarlyExercise = true` permits exercise prior to expiry.
- **European-style:** `allowEarlyExercise = false` permits exercise only at (or after) expiry.

Repayment timing flexibility is similarly explicit:

- `allowEarlyRepay = true`: repay at any time,
- `allowEarlyRepay = false`: repay only within allowed windows.

3.8.5 Callable Covenant (Opt-In)

Equalis Direct supports an optional lender-call covenant:

$$\text{allowLenderCall} \in \{0, 1\}.$$

If enabled and accepted, the lender may accelerate the agreement by setting the effective due time to “now” under protocol rules, forcing an earlier repay-or-exercise decision. If not accepted, the lender cannot unilaterally accelerate absent default.

3.8.6 Lifecycle and Timing Windows

Synthetic options follow the Direct lifecycle:

1. **Offer posted** (lender or borrower).
2. **Agreement accepted** (counterparty fills).
3. **Active period** (borrower holds the borrowed asset, collateral locked).
4. **Expiry decision** (repay or exercise).
5. **Recovery after grace** (if the borrower does nothing).

A typical configuration includes a grace window after `dueTimestamp` during which repay/exercise is still allowed before deterministic recovery becomes available.

3.8.7 Synthetic Call and Synthetic Put Constructions

Let Q be the option size (in underlying units) and K the strike (in quote units per underlying).

Synthetic Call on Underlying U with Strike K . Borrow the underlying and lock quote collateral:

- `borrowAsset` = U ,
- `principal` = $Q U$,
- `collateralAsset` = Qte ,
- `collateralLockAmount` = $K \cdot Q Qte$.

At expiry:

- if market price $> K$, borrower exercises (keeps $Q U$, forfeits KQ quote),
- if market price $< K$, borrower repays (returns $Q U$, recovers collateral).

Synthetic Put on Underlying U with Strike K . Borrow quote and lock underlying collateral:

- `borrowAsset` = Qte ,
- `principal` = $K \cdot Q Qte$,
- `collateralAsset` = U ,
- `collateralLockAmount` = $Q U$.

At expiry:

- if market price $< K$, borrower exercises (keeps quote, forfeits underlying),
- if market price $> K$, borrower repays (returns quote, recovers underlying).

These are **physically settled** payoffs: assets are exchanged; there is no cash settlement and no oracle-defined payout.

3.8.8 Offer Posting and Indexable Discovery

Synthetic options inherit the Direct offer model:

- **Lender-posted offers:** the lender posts option-writing terms (collateral ratio encodes strike, premium encodes price, duration encodes expiry).
- **Borrower-posted offers:** the borrower posts desired terms, and any lender may fill.

Offer and lifecycle events (posted, cancelled, accepted, repaid, exercised, recovered, called) are emitted and can be indexed, enabling transparent discovery of strikes, premia, expiries, and volumes.

3.8.9 Collateral Handling, Encumbrance, and Fee Routing

Collateral is locked via the unified encumbrance system and remains tracked inside the Position NFT accounting domain.

On **exercise** or **default**, collateral distribution is explicit and deterministic:

- the lender receives a configured share (compensation as the option writer),
- the remainder can be routed through the centralized fee router into:
 - Treasury,
 - Active Credit Index,
 - and the relevant pool Fee Index.

This makes synthetic options a fee-producing surface that integrates with system-wide value rails rather than an isolated derivatives product.

3.8.10 Packaging and Secondary Markets via the Position NFT

Synthetic options are agreement-native. They do not require issuing a separate option token to be transferable as an economic object, because the **Position NFT itself is a transferable container** for the entire balance sheet.

A Position NFT may accumulate a portfolio of option-like agreements over time (multiple expiries, implied strikes, and underlyings). Transferring the Position NFT transfers control of:

- locked collateral and remaining free balances,
- the set of active agreements (as buyer or writer),
- accrued index claims tied to the position,
- the right to repay, exercise, or allow default under the agreement rules.

This enables a secondary-market pattern where a user can sell an options portfolio by selling the Position NFT that contains it, *without minting option tokens*. This is complementary to option tokenization: it is suited to heterogeneous books, structured positions, and balance-sheet transfers.

Transferability must remain compatible with protocol invariants. In practice, transfers may be restricted during sensitive state transitions or otherwise conditioned to preserve correctness and ensure obligations transfer with the position.

3.8.11 What Synthetic Options Are (and Are Not)

They are:

- non-oracle, ratio-defined strikes,
- non-recourse for the borrower (collateral is the maximum loss),
- physically settled,
- customizable exercise style and covenants (including opt-in callability),
- naturally integrated with P2P offer discovery and Position NFT accounting,
- optionally transferable as a **portfolio** via Position NFT transfer.

They are not:

- a claim that options are risk-free (writers take directional and volatility risk),
- an oracle-priced cash-settled derivative,
- inherently fungible or easily splittable into standardized lots without additional tokenization,
- a guarantee of deep secondary liquidity (secondary markets depend on buyers and market structure).

If fungibility, partial transfers, or standardized lots are required, the system may also support explicit option tokenization (e.g., ERC-1155-based options). Synthetic options are the lightweight agreement-native primitive optimized for OTC-style deals, structured hedging, and minimizing dependency on external price feeds.

Tokenized derivatives (implementation). The current implementation includes tokenized options and futures series as separate modules. These instruments are physically settled and integrate with the same pool/encumbrance infrastructure but are not the primary focus of this paper’s core credit architecture.

4 EqualIndex: Deterministic Basket Tokens and Basket Flash Loans

EqualIndex is a permissionless index token primitive designed to create **static, fully backed basket tokens** and to expose that basket as a deterministic execution surface for multi-leg routing and arbitrage. EqualIndex is intentionally **boring in the right ways**: fixed composition, transparent backing, deterministic mint/burn, and fee flows that are explicit and auditable.

Creation gating (implementation). In the current implementation, “permissionless” creation is conditional: index creation can be enabled or disabled by configuration and may require a fixed creation fee. Creation also requires that all underlying assets already have pools, and the protocol automatically creates a dedicated pool for the index token itself to keep index units inside the same pool accounting system.

EqualIndex is integral to the Equalis system for two reasons:

1. it creates composable basket exposure that can be held inside Position NFTs, and
2. it produces fee flow from mint/burn and basket flash loans that is routed into system-wide fee rails (Treasury, Active Credit Index, and underlying pool Fee Index) while also rewarding index holders via fee pots.

4.1 Fixed-Composition, Fully Backed Baskets

An EqualIndex instance defines a set of underlying assets:

$$\mathcal{U} = \{T_1, T_2, \dots, T_k\},$$

and fixed per-index-unit quantities:

$$\mathbf{q} = (q_1, q_2, \dots, q_k),$$

where q_i is the amount of underlying token T_i backing one index unit (on a 10^{18} index unit scale).

No Rebalancing, No Oracles. EqualIndex does not rebalance and does not require an external price oracle to define “NAV”. The index unit is defined deterministically as a bundle. If users desire a different basket, they create a different index.

4.2 Minting

To mint x index units, a minter must deposit the exact proportional bundle:

$$\text{Deposit}(T_i) = x \cdot q_i \quad \forall i \in \{1, \dots, k\}.$$

These deposited underlyings are placed into the index vault as backing.

Mint Fees. For each underlying T_i , minting can collect a fee $f_i^{\text{mint}}(x)$ denominated in T_i . Each such fee is split into:

- a **pool share** routed through the system fee router for T_i (Treasury, Active Credit Index, and the T_i pool Fee Index), and
- an **index-holder share** added to the index’s T_i fee pot.

The implementation allows separate configurable fee-share parameters for mint/burn versus flash loan fees.

4.3 Burning

To burn x index units, the burner redeems a proportional share of vault backing and a proportional share of the accumulated fee pots.

Let:

- V_i be the current vault backing balance of underlying T_i ,
- P_i be the current fee pot balance of underlying T_i ,
- S be the total index supply.

Then burning x units returns:

$$\text{RedeemVault}(T_i) = \frac{x}{S} \cdot V_i, \quad \text{RedeemPot}(T_i) = \frac{x}{S} \cdot P_i,$$

for each underlying T_i (minus any configured burn fees).

Burn Fees. As with minting, burn fees are collected per underlying and split between:

- the pool share (system fee router) and
- the index-holder fee pots.

4.4 Fee Pots and Index Holder Economics

EqualIndex maintains per-underlying fee pots:

$$P_i \text{ for each } T_i \in \mathcal{U}.$$

These fee pots accumulate the index-holder share of fees (mint, burn, flash loans, and any other configured index fee sources). Index holders do not receive streaming yield; they receive value **realized on redemption**. This has two intended properties:

- **deterministic accounting:** the claim is proportional and auditable, and
- **no separate staking games:** holders are not required to chase schedules or claim emissions.

4.5 Basket Flash Loans

A distinguishing feature of EqualIndex is **basket flash loans**. Instead of borrowing a single asset, a borrower can atomically borrow the proportional amounts of every underlying asset corresponding to x index units.

For a flash loan of size x index units, the borrower receives:

$$\text{FlashBorrow}(T_i) = x \cdot q_i \quad \forall i \in \{1, \dots, k\}.$$

Within the same transaction, the borrower must return:

$$\text{FlashRepay}(T_i) = x \cdot q_i + \phi_i(x),$$

where $\phi_i(x)$ is the flash fee in underlying token T_i .

Intended Use: Deterministic Multi-Leg Execution and Arbitrage. Basket flash loans enable multi-leg routes without requiring the borrower to source each underlying asset separately. This is intended to serve as a deterministic execution device for:

- basket arbitrage (index bundle versus external venues),
- multi-asset rebalancing or routing operations,
- atomic strategies that require temporary access to the full basket.

The protocol does not assume the strategy is profitable; it provides a deterministic mechanism that can be used when external price discrepancies exist.

4.6 Fee Routing: System Rails and Index Rails

For each underlying token T_i , the fee $\phi_i(x)$ collected during a basket flash loan is split into:

- **System share:** routed through the fee router for token T_i into Treasury, Active Credit Index, and the T_i pool Fee Index.
- **Index-holder share:** added to the index fee pot P_i .

Mint and burn fees follow the same conceptual split.

This is a key system property: EqualIndex is not an isolated product with siloed incentives. It is a fee-producing surface that:

1. rewards index holders via fee pots, and
2. strengthens the unified per-token pools and indexes for each underlying asset.

4.7 Integration With Position NFTs and Encumbrance

EqualIndex supports interactions via Position collateral. In addition to external wallet mint/burn, a position may allocate underlying assets to index minting (and receive index units) via internal accounting. This leverages the encumbrance model:

- underlying assets remain within the unified pool accounting domain,
- the position's balances are marked as encumbered for index backing,
- index units and associated claims are recorded under the Position NFT.

This integration enables users to hold basket exposure and participate in index-related fee flows without abandoning pool-level fee rails.

4.8 What EqualIndex Is (and Is Not)

EqualIndex is:

- permissionless basket creation when enabled (subject to system constraints and creation fees),
- fixed-composition, fully backed baskets,
- deterministic mint/burn accounting,
- basket flash loans for atomic multi-asset execution,
- fee routing that rewards index holders and feeds system-wide per-token indexes.

EqualIndex is not:

- an actively managed fund,
- a rebalancing index with governance-driven weight changes,
- an oracle-priced NAV product,
- a yield guarantee.

5 Unified Liquidity and Encumbrance

Equalis is designed to eliminate a common DeFi tradeoff: increasing product surface area typically fragments liquidity and forces users to choose which venue gets their capital today. Equalis instead enforces a **canonical base pool per token** and treats internal venues as accounting-level allocations of that pool rather than separate liquidity systems.

5.1 One Pool per Token (Globally)

For each token T , the protocol maintains a canonical base pool $\mathcal{P}_T$. All core modules that use T reference the same pool state and the same fee rails.

Managed pools (implementation). In addition to the base pool, the current codebase supports managed pools for the same underlying asset. Managed pools can enforce whitelist membership and custom parameters while routing a configurable system share of fees back to the base pool (or treasury). This allows curated pool variants without abandoning unified fee rails at the base layer.

This yields two immediate properties:

- **No liquidity duplication:** new modules do not require new T liquidity silos.
- **No incentive islands:** fees in T can be routed back into the same per-token indexes regardless of which module generated them.

5.2 Internal Ledger Model

Most activity in Equalis is represented as internal ledger moves rather than external token transfers. Conceptually, deposits into $\mathcal{P}_T$ create a ledger balance for a Position NFT, and subsequent actions modify that ledger balance subject to deterministic constraints.

This design is intended to:

- reduce unnecessary token transfers between contracts,
- enable coherent accounting across modules,
- keep liquidity unified for system features (including flash liquidity where applicable).

5.3 Encumbrance: Allocation Without Exit

Encumbrance is the protocol mechanism that allocates liquidity from a position into a module *without removing it from the pool's accounting domain*. When a position allocates an amount x of token T to a module M , the protocol records:

- a reduction in the position's *free* balance of T by x ,
- an increase in the position's *encumbered* balance of T by x ,
- a module-specific encumbrance record under M .

Encumbrance namespaces (implementation). The implementation distinguishes explicit encumbrance buckets, including `directLocked` (borrower collateral), `directLent` (maker inventory / lent capital), `directOfferEscrow` (offer escrow), and `indexEncumbered` (index backing). These namespaces affect solvency and Active Credit eligibility differently and are tracked per position and per pool.

Let a position p have ledger balances for token T :

$$\begin{aligned} B_{p,T}^{\text{free}} &:= \text{free (unencumbered) balance,} \\ B_{p,T}^{\text{enc}} &:= \text{encumbered balance.} \end{aligned}$$

An encumbrance action of size x updates:

$$B_{p,T}^{\text{free}} \leftarrow B_{p,T}^{\text{free}} - x, \quad B_{p,T}^{\text{enc}} \leftarrow B_{p,T}^{\text{enc}} + x,$$

and records (p, T, M, x) in module state.

Strict Non-Double-Use. Encumbrance is not rehypothecation. The encumbered amount cannot simultaneously be used for conflicting obligations. Once marked encumbered, it is unavailable for:

- withdrawal,
- additional encumbrance to other modules,
- serving as the same unit of free liquidity for other actions,

until it is explicitly released by the module according to deterministic rules.

5.4 Encumbrance Categories

Encumbrance can represent different kinds of allocation depending on the module:

- **Market reserves:** inventory allocated to AMM auction reserves (solo or community).
- **Direct agreements:** collateral locked under peer-to-peer credit agreements.
- **Index backing:** underlying assets committed to EqualIndex backing flows or held for basket-related operations.
- **Other modules:** any module that consumes inventory can define its own encumbrance namespace.

The unified encumbrance interface allows modules to be composed while maintaining a canonical base pool per token (and consistent fee rails across optional managed pools).

5.5 Why Unified Liquidity Matters

The one-pool-per-token model changes how the system scales:

- **More modules can mean more fee sources, not more fragmentation.**
- **Users can move strategies internally without abandoning pool exposure.**
- **System incentives can follow activity.** Fee routing can remain coherent because every module settles into the same per-token accounting rails.

In conventional designs, a user who moves liquidity to a new venue often stops earning from their previous venue. In Equalis, module allocation is represented as an internal accounting transition rather than an exit, so the user can maintain consistent exposure to pool-level fee rails while deploying capital into additional fee-producing surfaces (subject to the index earning weight rules defined in the next section).

5.6 Composability and Extensibility

Because encumbrance is an accounting mechanism, new modules can be added without requiring new per-token liquidity pools. A compliant module must:

1. allocate and release inventory via encumbrance,
2. settle deterministically using onchain rules,
3. route fees through the per-token fee router.

This preserves the core system invariant: **liquidity is unified by token, while risk is isolated by module design and explicit agreements.**

The next section specifies how value is routed back to participants through the dual index system and how earning weights are defined for passive and active participation.

6 Dual Index System: Fee Index and Active Credit Index

Equalis routes value back to participants through a dual index system. The goal is to reward passive liquidity without starving the capital that is actively deployed. Many DeFi designs conflate these behaviors into a single reward stream, which makes incentives noisy and easy to game. Equalis separates them into two monotonic per-token indexes:

$$\begin{aligned} I_T &:= \text{Fee Index for token } T \quad (\text{passive leg}), \\ A_T &:= \text{Active Credit Index for token } T \quad (\text{active leg}). \end{aligned}$$

For each token T , fees generated anywhere in the system *in token* T can be routed into I_T and/or A_T through the pool fee router. This allows multiple modules to feed the same per-token value rails without creating separate incentive islands.

6.1 Index Accounting Model

Indexes are accounting devices that allow the protocol to distribute fees without iterating across all positions on each fee event. A position p maintains per-token snapshots:

$$\text{lastFeeIndex}_{p,T}, \quad \text{lastActiveIndex}_{p,T},$$

and claimable amounts are computed from the difference between current global index values and the position's last snapshots.

6.1.1 Global Index Updates

For token T , let:

- F_T^{FI} be the amount of token T routed to the Fee Index during some event or accounting interval,
- F_T^{ACI} be the amount of token T routed to the Active Credit Index during the same.

Define the total earning weights:

$$W_T^{FI} = \sum_p w_{p,T}^{FI}, \quad W_T^{ACI} = \sum_p w_{p,T}^{ACI},$$

where $w_{p,T}^{FI}$ and $w_{p,T}^{ACI}$ are the per-position earning weights for the Fee Index and Active Credit Index respectively.

Then index increments are:

$$\Delta I_T = \frac{F_T^{FI}}{W_T^{FI}}, \quad \Delta A_T = \frac{F_T^{ACI}}{W_T^{ACI}},$$

with the convention that if an earning weight denominator is zero, the corresponding routed fees are retained according to protocol rules (e.g., held in a buffer) until distribution is possible.

6.1.2 Position-Level Accrual

A position's newly accrued claim from the Fee Index is:

$$\Delta y_{p,T}^{FI} = w_{p,T}^{FI} \cdot (I_T - \text{lastFeeIndex}_{p,T}),$$

and similarly for the Active Credit Index:

$$\Delta y_{p,T}^{ACI} = w_{p,T}^{ACI} \cdot (A_T - \text{lastActiveIndex}_{p,T}).$$

When the position is updated (on deposit, borrow, encumbrance changes, or explicit claim), the protocol advances the snapshots:

$$\text{lastFeeIndex}_{p,T} \leftarrow I_T, \quad \text{lastActiveIndex}_{p,T} \leftarrow A_T.$$

6.2 Fee Index (Passive Leg)

The Fee Index is intended to reward passive liquidity provision in a way that remains coherent even when users borrow against themselves or allocate liquidity into modules.

6.2.1 Earning Weight for the Fee Index

For each position p and token T , define:

- $\text{Deposit}_{p,T}$: total deposited ledger balance in pool $\mathcal{P}_T$,
- $\text{Debt}_{p,T}$: same-asset debt (self-secured credit) in token T .

A simple, conservative representation of the Fee Index earning weight is net equity:

$$w_{p,T}^{FI} = \max(\text{Deposit}_{p,T} - \text{Debt}_{p,T}, 0).$$

This captures the core intent: a position should not be able to borrow T from the pool and then count the borrowed units as additional passive principal for Fee Index earning in T .

Encumbered Capital and the Fee Index. Encumbered balances remain within the pool accounting domain. Whether encumbered balances contribute to $w_{p,T}^{FI}$ is a protocol design choice that can be made explicit as:

$$\text{Deposit}_{p,T} = B_{p,T}^{\text{free}} + B_{p,T}^{\text{enc}},$$

with $w_{p,T}^{FI}$ computed on the total deposit side and normalized by same-asset debt. This preserves the concept that participating in modules does not necessarily forfeit passive exposure, while still preventing borrow-deposit double counting.

Maintenance and negative yield (implementation). The current implementation applies a pool-level maintenance fee that reduces principal over time via a negative maintenance index. Fee Index settlement applies maintenance reductions before positive yield is accrued.

Backing and yield reserves (implementation). Fee Index accruals are backed by a pool-level yield reserve and enforce availability constraints against tracked balances plus active credit principal. This preserves solvency even when liquidity is encumbered or lent out.

6.3 Active Credit Index (Active Leg)

The Active Credit Index rewards deployed capital: borrowing, underwriting, and module participation that puts capital to work.

6.3.1 What Counts as Active

Active weight $w_{p,T}^{ACI}$ is designed to include eligible deployed capital such as:

- same-asset borrowed principal in token T (self-secured credit),
- capital locked or allocated under peer-to-peer credit agreements,
- capital encumbered into approved fee-producing modules (e.g., AMM auction inventory, index-related allocations),
- other protocol-defined active uses.

The precise active categories are defined by module interfaces, and the system can restrict which encumbrance namespaces count as active to ensure ACI rewards align with fee-producing behavior.

6.3.2 Time-Gating and Weighted Maturity (Implementation Detail)

The current implementation enforces a time gate (24 hours) before active balances contribute to $w_{p,T}^{ACI}$. Increases in active principal are blended via weighted start-time dilution to reduce short-term cycling advantages. Active credit is tracked in separate states for *debt* and *encumbrance*, each with its own maturity clock and index snapshot.

6.3.3 Anti-Gaming Constraints

Because ACI rewards activity, it must defend against trivial short-lived cycling. Equalis enforces anti-gaming constraints such as:

- **time gating:** active balances must mature for a minimum duration before being counted toward $w_{p,T}^{ACI}$,
- **weighting rules:** sudden large increases in active balance are blended to prevent “warm-up with dust then scale” exploitation,
- **eligibility constraints:** only approved activity types count toward ACI weight.

These controls are not emissions-like complexity; they are accounting rules intended to preserve fairness between short-term tactics and long-term participation.

6.4 Fee Sources and Routing Into Indexes

Protocol surfaces generate fees in many ways, including but not limited to:

- AMM/community auction swap fees (maker share plus protocol remainder routed by fee router),
- MAM curve fill fees (maker share plus protocol remainder routed by fee router),
- peer-to-peer agreement fees and penalties (including default-related flows),
- EqualIndex mint and burn fees (per underlying asset),
- EqualIndex basket flash loan fees (per underlying asset),
- atomic settlement swap fees (where configured),
- pool action fees (borrow/repay/withdraw/flash/close),
- other module-defined fees that settle into pool tokens.

For each token T , fees collected in T are routed through the token’s fee router and split according to configured rules, typically including:

- a **direct participant share** (e.g., makers),
- a **protocol remainder** routed by configuration into Treasury, Active Credit Index, and Fee Index rails.

Maintenance fees (implementation). Pool-level maintenance fees are accrued separately and paid to a designated receiver; they reduce depositor principal via the maintenance index and are not routed through I_T or A_T .

6.5 System-Level Implication: Expanding Surfaces Without Fragmenting Yield

Because the Fee Index and Active Credit Index are per-token and global, adding a new module that generates fees in token T can increase F_T^{FI} and/or F_T^{ACI} without requiring a new T liquidity silo.

In this model, increased protocol surface area does not inherently dilute yield by fragmenting liquidity. Instead, new fee sources can strengthen the same per-token indexes. Yield per unit depends on realized fee volume relative to the earning weight denominators, but **venue expansion does not require splitting liquidity into separate incentive islands.**

7 AMM Auctions: Solo and Community

Equalis includes auction-style AMM venues designed to provide onchain execution surfaces without fragmenting liquidity or incentives. These venues allow positions to deploy inventory into markets while remaining inside the unified pool accounting domain via encumbrance. Swap activity can pay makers directly while also routing value into the per-token Fee Index and Active Credit Index.

The intent is simple: **markets should be fee-producing surfaces that feed the same system rails**, rather than standalone products that require separate liquidity bootstrapping and separate incentive programs.

7.1 Design Goals

AMM Auctions are designed to:

- provide deterministic, permissionless execution surfaces,
- allow inventory allocation via encumbrance rather than pool exit,
- compensate makers directly for providing inventory and pricing,
- route a configured share of fees back to the unified per-token indexes,
- support both single-maker and multi-maker participation patterns.

7.2 Solo AMM Auctions

A *Solo AMM Auction* is created by a single Position NFT that allocates two-token reserves into an auction contract. Conceptually, it behaves like a time-bounded constant-product market where the maker supplies both sides of the pair.

7.2.1 Inventory Allocation

Let the maker position be p and the token pair be (X, Y) . The maker allocates:

$$R_X \text{ of token } X, \quad R_Y \text{ of token } Y,$$

by encumbering R_X in pool $\mathcal{P}_X$ and R_Y in pool $\mathcal{P}_Y$ under the auction namespace.

This updates the maker's free and encumbered balances deterministically and prevents double-use of the same units of liquidity.

7.2.2 Swaps and Fee Flow

A taker swap consumes one reserve and returns the other according to the auction's pricing rule. On each swap, the protocol can collect fees and split them into:

- a **maker share** paid to the auction maker,
- a **protocol remainder** routed through the per-token fee router (Treasury, Active Credit, and Fee Index according to configured splits).

When fees are collected in token T , routed protocol portions contribute to F_T^{FI} and F_T^{ACI} and update the global indexes for pool $\mathcal{P}_T$.

7.3 Community Auctions

A *Community Auction* allows multiple Position NFTs to contribute inventory to a shared auction. This enables maker participation without requiring a single maker to supply the full reserve.

7.3.1 Contribution and Shares

Makers contribute reserves in a fixed ratio aligned with current auction reserves. Contributions are represented by shares that preserve proportional ownership.

A common deterministic share rule is:

$$\text{shares} \propto \sqrt{\Delta R_X \cdot \Delta R_Y},$$

where ΔR_X and ΔR_Y are the maker's contributed amounts. The share system ensures that makers earn fees proportional to their contribution without requiring an oracle price.

7.3.2 Fees and Distribution

Community auction swap fees can be distributed to:

- makers pro-rata by shares (maker fee distribution),
- the per-token indexes (Fee Index and Active Credit Index) via fee routing,
- protocol allocations where configured.

This preserves the core principle: **makers are paid for making markets**, while the system as a whole benefits from market activity through shared index rails.

7.4 Integration With Unified Liquidity and Indexes

AMM Auctions are integrated into unified liquidity through encumbrance:

- maker inventory remains in the pool accounting domain as encumbered balances,
- swaps settle through deterministic accounting updates,
- fees can be routed into the same per-token indexes that reward passive and active participants.

This model is intentionally different from isolated AMMs that require separate liquidity pools and separate reward programs. Here, markets are composable surfaces that contribute to a single per-token liquidity substrate and a shared incentive architecture.

7.5 Active Participation Incentives

Capital encumbered into AMM Auctions is a form of deployed inventory. Under the Active Credit Index eligibility rules, such encumbered balances may contribute to $w_{p,T}^{ACI}$ for the relevant tokens, enabling makers to earn:

- maker fees from swaps, and
- index-based rewards (ACI and potentially Fee Index exposure subject to the earning weight rules).

This is not intended as a free lunch. It is an incentive alignment mechanism: when users allocate inventory that enables system activity, the system routes part of the resulting fee flow back to them through explicit rails rather than through inflationary emissions.

7.6 Summary

Solo and Community AMM Auctions provide execution venues that:

- avoid liquidity fragmentation by using encumbrance allocation,
- compensate makers directly for providing inventory,
- route fees into unified per-token indexes, strengthening system-wide value rails.

The next section formalizes the capital efficiency implications of this architecture and introduces a precise definition of protocol-perpetual leverage loops built from explicit credit and internal execution surfaces.

8 Capital Efficiency, Redefined

Equalis is designed around a different notion of capital efficiency than what is typical in DeFi. In many systems, “capital efficiency” is achieved through some combination of:

- rehypothecation or recursive leverage,
- liquidation-based credit expansion,
- liquidity fragmentation across venues with separate incentive programs.

Equalis instead defines capital efficiency in terms of **utilization without fragmentation**:

Capital efficiency is the fraction of deposited capital that can be allocated into fee-producing activity while remaining inside a single per-token pool and retaining coherent exposure to system-wide fee routing.

This definition is intentionally constrained. It does not claim that capital can be safely multiplied without bound. It claims that capital can be *deployed more flexibly* within a unified accounting system, so that additional activity tends to increase aggregate fee flow *without* requiring users to abandon one venue to participate in another.

8.1 Unified Pools Prevent Incentive Dilution by Fragmentation

In fragmented architectures, adding new products often creates new liquidity silos and new incentive islands. Users move capital to chase yield, and the system expends continuous subsidies to keep liquidity in each silo.

Equalis enforces a canonical base pool per token (with optional managed pools). This changes the scaling dynamic:

- new modules do not create a new pool for token T ,
- fees collected in token T route back into the same per-token indexes (I_T, A_T) ,
- users can reallocate capital internally without leaving the pool accounting domain.

The result is not guaranteed yield; it is **coherent yield**. Activity in any module that collects fees in token T can strengthen the same fee rails that reward pool participants in token T .

8.2 A Simple “No Fragmentation Dilution” Argument

For token T , let:

- F_T^{FI} be total fees in T routed to the Fee Index over some interval,
- W_T^{FI} be the total Fee Index earning weight,
- F_T^{ACI} be total fees in T routed to the Active Credit Index,
- W_T^{ACI} be the total Active Credit Index earning weight.

Index increments are:

$$\Delta I_T = \frac{F_T^{FI}}{W_T^{FI}}, \quad \Delta A_T = \frac{F_T^{ACI}}{W_T^{ACI}}.$$

Now consider adding a new module M that generates additional fees in token T and routes an index share:

$$F_T^{FI} \leftarrow F_T^{FI} + \delta_T^{FI}(M), \quad F_T^{ACI} \leftarrow F_T^{ACI} + \delta_T^{ACI}(M).$$

Crucially, the module does not create a separate pool, so it does not require partitioning liquidity into a new earning base. The fee rails remain global per token. Therefore, the system avoids a common dilution mechanism: *splitting the same liquidity across multiple competing reward systems*.

Yield per unit can still decrease if earning weights grow faster than fee flow (more capital competing for the same activity), but this is a normal economic effect. What is avoided is dilution caused purely by **venue fragmentation**.

8.3 Active Capital Can Earn Without Exiting the Pool

Encumbrance allows capital to be allocated into fee-producing surfaces while remaining within the pool accounting domain. Depending on the earning weight rules:

- passive exposure can remain coherent (Fee Index based on net equity),
- active exposure can be earned for deployed balances (Active Credit Index based on eligible active categories).

This enables strategies where capital is simultaneously:

1. contributing to system liquidity (as a depositor in $\mathcal{P}_T$),
2. deployed into a module (as encumbered inventory),
3. participating in explicit credit (as borrower or lender),

without requiring the user to abandon system-wide fee rails to participate in new activity.

8.4 EqualIndex as an Efficiency Surface

EqualIndex reinforces this capital efficiency model:

- basket flash loans create a deterministic multi-leg execution surface that can generate fee flow,
- fees are collected per underlying and routed both to index holder fee pots and to system-wide fee rails (Treasury, ACI, underlying pool Fee Index),
- basket activity therefore strengthens unified pools and indexes for each underlying asset rather than creating an isolated incentive island.

In other words, EqualIndex is not merely “an index token.” It is a fee-producing execution primitive whose fee routing is designed to reinforce unified per-token liquidity.

8.5 What This Is Not Claiming

Because the system enables more ways to deploy capital, it is important to be explicit about what is *not* implied:

- **No guarantee of superior returns.** Returns depend on realized fee volume and usage.
- **No elimination of risk.** Peer-to-peer credit includes counterparty risk; modules can introduce domain-specific risks.
- **No free multiplication of collateral.** Encumbrance is strict accounting, not double-use. The same units of liquidity cannot satisfy multiple conflicting obligations.

8.6 Summary

Equalis redefines capital efficiency as utilization within a unified accounting system:

- a canonical base pool per token prevents liquidity and incentives from fragmenting into silos (while allowing optional managed pools with system-share routing),
- encumbrance enables internal deployment without pool exit,
- dual indexes keep passive and active incentives legible,

- modules (including EqualIndex and AMM auctions) expand fee sources while feeding the same per-token rails.

The next section defines protocol-perpetual leverage loops and presents a concrete example of how explicit credit and internal execution surfaces can be composed into continuity-preserving strategies.

9 Perpetual Leverage Loop

This section defines a precise notion of “perpetual leverage” in Equalis and explains why it is structurally different from liquidation-based leverage in typical DeFi money markets.

9.1 Definition: Protocol-Perpetual Leverage

In this paper, “perpetual” is a statement about **protocol behavior**, not about guaranteed profitability, infinite scaling, or immunity to poor execution. A leverage loop is **protocol-perpetual** if the protocol contains no rule that forces collateral sale solely due to mark-to-market price changes.

Definition (Protocol-Perpetual Leverage). A leveraged position is protocol-perpetual if it can remain open across time without any oracle-triggered liquidation mechanism or automatic collateral sale conditioned only on price movement. Continuation is determined by explicit contractual obligations (e.g., scheduled payments, agreed covenants) and voluntary user actions (repayment/unwind).

This distinguishes Equalis from systems where leverage is maintained by continuously satisfying a health factor that is computed from external price feeds, and where failure results in forced third-party liquidation.

9.2 Why This Matters

In liquidation-based systems, leverage is reactive:

- volatility can force a close at the worst moment,
- temporary drawdowns become realized losses,
- liquidation pathways attract adversarial competition and MEV extraction.

In Equalis, leverage continuity can be expressed through explicit agreements that do not require price-triggered liquidation. Risk is not removed; it is **made legible**:

- if the borrower meets explicit payment obligations, the position persists,
- if the borrower defaults, enforcement follows deterministic agreement rules,
- if the borrower accepted a callable covenant, the lender may accelerate under those terms,
- if the borrower did not accept callability, the lender cannot unilaterally terminate absent default.

9.3 Canonical Loop Construction

A canonical loop composes three surfaces:

1. **Explicit credit** (Equalis Direct) to borrow an asset the user does not own,
2. **Execution** (AMM Auction or other routing venue) to convert borrowed inventory,
3. **Unified pooling** (deposit into the relevant pool) to increase the user's exposure and/or collateral base.

9.3.1 Loop Outline

Consider two tokens:

- T_1 : the asset the borrower wants to accumulate exposure to,
- T_2 : the asset the borrower borrows from a lender.

Let the borrower begin with an initial deposit of T_1 into $\mathcal{P}_{T_1}$.

Each loop iteration:

1. Borrow Δ units of T_2 via an Equalis Direct agreement (term or rolling).
2. Swap Δ units of T_2 into T_1 using an internal execution surface (e.g., AMM Auction).
3. Deposit received T_1 back into $\mathcal{P}_{T_1}$, increasing the position's T_1 balance.

The position's exposure to T_1 grows with each iteration while obligations are governed by the explicit agreement schedule (e.g., a predictable monthly payment).

9.4 Termination Conditions (What Ends the Loop)

A protocol-perpetual loop is not immortal. It persists until one of the following occurs:

- **Borrower unwind:** the borrower voluntarily repays and closes (partially or fully) the agreement(s).
- **Borrower default:** the borrower fails explicit obligations (e.g., missed payment beyond grace), triggering deterministic default consequences.
- **Lender call (only if accepted):** if the borrower accepted `allowLenderCall = 1` on a given agreement, the lender may accelerate under protocol rules; if not accepted, this termination path does not exist.

Notably absent is a fourth condition common in liquidation systems:

There is no termination condition of the form “price moved, therefore forced collateral sale.”

9.5 Feasibility Versus Forced Unwind

Market conditions can affect the *feasibility* or attractiveness of entering or scaling the loop (e.g., execution price impact, available posted terms), but these conditions do not produce protocol-level forced unwinds in the deterministic model described here.

This distinction is central:

- **Feasibility constraints** determine whether a strategy is worth initiating.
- **Forced unwind rules** determine whether a position is automatically closed.

Equalis aims to remove forced unwind rules based solely on price while keeping feasibility grounded in reality.

9.6 Index Incentives During the Loop

Because Equalis tracks both passive and active participation:

- the borrower’s deposited equity in $\mathcal{P}_{T_1}$ can maintain Fee Index exposure (subject to net equity weighting),
- active deployment (borrowed principal, locked collateral, and eligible encumbered balances) can earn Active Credit Index exposure according to system rules,
- execution fees and agreement fees can route value back into the per-token indexes for the relevant assets.

This incentive model is designed to reward both the capital that provides liquidity and the capital that puts the system to work.

9.7 Summary

Protocol-perpetual leverage in Equalis is achieved by replacing oracle-triggered liquidation thresholds with explicit contractual obligations and deterministic enforcement. A leverage loop can persist through volatility because continuation is not conditioned on maintaining a health factor. The loop ends only by voluntary unwind, explicit default, or opt-in callability where agreed.

The next section formalizes risk and threat considerations, including smart contract risk, peer-to-peer credit default risk, and the role of module isolation in maintaining coherent system behavior.

10 Maker Auction Market (MAM) Curves

Maker Auction Markets (MAM) provide a one-sided execution venue where a Position NFT holder (the maker) sells a **base asset** for a **quote asset** using a **time-varying price curve**. Each curve is a **linear Dutch auction** with an explicit start time, duration, price schedule, and maximum sell volume. Unlike constant-product AMMs, a MAM curve does not derive price from reserves or from the size of the most recent trade. Price is predetermined by the maker and evolves linearly over time.

10.1 Core Properties

A MAM curve has the following defining characteristics:

- **Time-bounded:** a curve has a start time and duration, and expires at a deterministic end time.
- **One-sided liquidity:** the maker locks base asset inventory and receives quote asset as fills occur.
- **Volume-limited:** the maker specifies a maximum base volume to sell, and fills reduce remaining volume until exhausted.
- **Updatable pricing:** the maker can update pricing parameters while the curve is active.
- **Batchable operations:** curves support batch create, batch update, and lifecycle operations that are amenable to higher-frequency maker behavior.
- **Event-indexable discovery:** creation, updates, fills, cancellation, and expiration emit events that can be indexed into an order-book-like UI.

Current implementation constraints. The deployed code currently constrains MAM curves to quote-per-base pricing, fee asset = token-in, and a fixed generation scheme at creation; updates require future start times. These constraints can be relaxed in later versions but should be treated as part of the current specification.

10.2 Base and Quote Assets

Each curve is defined over a token pair (T_A, T_B) and a side:

- **side = false** means **sell T_A for T_B** (base is T_A , quote is T_B).
- **side = true** means **sell T_B for T_A** (base is T_B , quote is T_A).

Prices are expressed as **quote per base** with 10^{18} precision:

$$\text{price} = \frac{\text{quoteAmount}}{\text{baseAmount}} \times 10^{18}.$$

10.3 Price Curves as Orders

A MAM curve generalizes common order types through a single mechanism:

Descending Curve (Classic Dutch Auction). If $\text{startPrice} > \text{endPrice}$, price decreases linearly over time. Takers may wait for a better price, but risk being filled by others first.

Ascending Curve (Reverse Dutch Auction). If $\text{startPrice} < \text{endPrice}$, price increases linearly over time. This supports makers who prefer early execution at a discount and progressively worse pricing later.

Flat Curve (Limit Order). If $\text{startPrice} = \text{endPrice}$, the price is constant for the entire duration. This acts like a **flat-price auction** and is directly interpretable as a **limit order**:

- the maker offers up to **maxVolume** base at a fixed price,
- fills occur at exactly that price until remaining volume is exhausted or the curve expires.

10.4 Pricing Model (Linear Interpolation)

Let:

- t_0 be **startTime**,
- D be **duration**,
- $t_1 = t_0 + D$ be the end time,
- P_0 be **startPrice**,
- P_1 be **endPrice**.

The current price $P(t)$ is:

$$P(t) = \begin{cases} P_0 & \text{if } t \leq t_0, \\ P_1 & \text{if } t \geq t_1, \\ P_0 + (P_1 - P_0) \cdot \frac{t-t_0}{D} & \text{otherwise.} \end{cases}$$

This schedule is deterministic and does not depend on fill size or on prior fills, except for remaining volume constraints.

10.5 Fill Mechanics

A taker specifies `amountIn` (quote net amount), `maxQuote` (quote cap including fees), `minOut` (base received), and `deadline`. The protocol computes the current price and determines base out:

$$\text{baseOut} = \frac{\text{amountIn} \cdot 10^{18}}{P(t)}.$$

Constraints enforced include:

- `baseOut` must be non-zero,
- `baseOut` must not exceed remaining base volume,
- required quote transfer (`amountIn` + fee) must respect `maxQuote`,
- `baseOut` must be at least `minOut`,
- the current timestamp must not exceed `deadline`.

Curves support partial fills. Multiple takers can fill the same curve until remaining volume reaches zero, at which point the curve deactivates.

10.6 Fee Structure and Routing

Each fill can charge a fee (basis points on `amountIn`, denominated in the quote asset). The implementation applies:

- **Maker share:** configurable via `mamMakerShareBps`,
- **Protocol remainder:** routed through the fee router into Treasury, Active Credit Index, and Fee Index rails.

This fee model allows MAM to compensate direct market makers while also feeding system-wide value rails.

10.7 MEV Resistance (Relative to Constant-Product AMMs)

MAM curves are designed to reduce common MEV vectors that rely on **price impact** and **trade-dependent repricing**:

- **No reserve-based price impact.** In a constant-product AMM, a taker trade changes reserves and therefore changes the marginal price, enabling sandwich patterns that extract value from induced slippage. In a MAM curve, the price at time t is determined by the curve schedule, not by the size of the previous trade.
- **Deterministic quoting.** A taker can query quote requirements via view functions (e.g., preview quote cap) and protect execution with `maxQuote`, `minOut`, and `deadline`.
- **Less room for slippage games.** Since fills do not move the curve price, there is less opportunity to profit from manipulating the taker's execution price via immediate pre-trades that shift the curve's pricing function.

This does not claim that MEV disappears. It claims that MAM reduces a dominant category of MEV that depends on trade-induced repricing. Competition among takers still exists (for example, racing to fill a favorable segment of a curve), but it is a simpler and more legible form of competition than adversarial slippage extraction.

10.8 Curves as High-Frequency Orders

MAM curves are designed to be **cheap to update** because mutable pricing parameters are stored separately from immutable curve identity. The maker can update:

`(startPrice, endPrice, startTime, duration)`

while the curve is active. Each update increments a **generation** counter and recomputes a commitment hash, which allows offchain systems to detect stale quotes and treat the curve as a versioned order.

Batch update functions allow makers to update multiple curves in one transaction, supporting higher-frequency adjustment patterns without requiring a new pool or a new venue.

10.9 Expressing Iceberg Liquidity

Although each curve has a fixed maximum volume, the system can simulate iceberg-style liquidity through composition:

- **Staged curves:** a maker posts a sequence of smaller-volume curves with staggered start times, revealing liquidity in tranches over time.
- **Rolling replenishment:** as one curve fills or expires, the maker creates a new curve with similar parameters, maintaining a persistent presence without exposing full inventory at once.
- **Many small flat curves:** a maker can approximate an iceberg limit book by posting multiple flat-price curves (each with modest `maxVolume`) rather than one large visible order.

Because create and update operations are batchable and because updates are cheap relative to rebuilding a full AMM position, MAM supports these liquidity-shaping strategies without complex rebalancing mechanics.

10.10 Discovery and Indexing

MAM includes explicit discovery surfaces that can be indexed into an order-book-like UI:

- by maker position (Position NFT),
- by token pair,
- by a global active-curve list,
- by curve status and quote functions.

Event emissions for `CurveCreated`, `CurveUpdated`, fills, cancellation, and expiration allow external indexers to reconstruct a transparent onchain market of outstanding maker curves.

10.11 Summary

MAM curves provide a deterministic market surface with:

- linear time-based pricing that can be descending, ascending, or flat (limit order),
- one-sided, volume-limited maker inventory,

- reduced reliance on trade-induced repricing, improving resistance to common slippage-based MEV,
- cheap and batchable updates for higher-frequency maker behavior,
- composable techniques for iceberg-like liquidity via staged or rolling curves,
- fee splits that pay makers while feeding system-wide Treasury, Active Credit, and Fee Index rails via the router.

11 Risk Model and Threat Model

Equalis is designed to reduce specific classes of risk common in DeFi (oracle-triggered forced unwinds, liquidation MEV incentives, and liquidity fragmentation). It does not eliminate risk. This section enumerates the major risk categories, the adversarial models assumed, and the architectural mechanisms used to constrain blast radius and preserve system invariants.

11.1 Threat Model Assumptions

Unless explicitly stated otherwise, the protocol assumes:

- **Adversarial users:** any address may behave strategically, including attempting to extract value at the expense of others.
- **Adversarial miners/validators/searchers:** transaction ordering is adversarial; MEV exists.
- **Adversarial counterparties:** in peer-to-peer credit, lenders and borrowers may default if economically rational.
- **Composability hazards:** external protocols and tokens may behave unexpectedly (reentrancy-like patterns, fee-on-transfer tokens, non-standard ERC20 behavior).
- **Implementation fallibility:** smart contracts can contain bugs; dependencies may fail; upgrades may be needed early in the lifecycle.

11.2 Risk Categories

11.2.1 Smart Contract and Implementation Risk

This is the dominant risk in early-stage protocols. The system contains multiple moving parts:

- unified pool accounting,
- Position NFT state and index snapshots,
- encumbrance bookkeeping across modules,
- fee routing and index updates,
- module-specific settlement (AMM auctions, MAM curves, EqualIndex mint/burn/flash loans, P2P agreements).

Mitigations.

- deterministic accounting surfaces with explicit invariants,
- modular architecture enabling isolation and targeted hardening,
- progressive freezing of stable surfaces (road to immutability),
- extensive invariant testing, fuzzing, and adversarial simulation.

11.2.2 Token and Integration Risk

Not all ERC20 tokens behave identically. Risks include:

- fee-on-transfer or rebasing behavior,
- non-standard return values,
- tokens with admin controls that can freeze or confiscate balances,
- blacklisting and pausing mechanisms.

Mitigations.

- explicit token allowlists or token-specific adapters (if used),
- conservative support policy for non-standard tokens,
- strict accounting assumptions and defensive transfer handling.

11.2.3 Peer-to-Peer Credit Default Risk

Equalis Direct exposes credit risk explicitly. Defaults are possible and expected in adversarial settings:

- borrowers may choose to default if it is economically favorable,
- lenders may attempt to structure terms that are predatory or mispriced,
- counterparties may attempt griefing or strategic timing around payment windows.

Mitigations.

- explicit contract terms, collateral requirements, and grace windows,
- deterministic default handling rather than liquidation auctions,
- optional covenants (including opt-in callability) to allow differentiated risk products,
- event-indexed offer flow enabling transparent market discovery and competition.

11.2.4 Liquidity and Execution Risk

Execution venues (AMM auctions, MAM curves, EqualIndex basket flash loans) are surfaces where external market conditions matter:

- low liquidity can increase price impact and reduce strategy feasibility,
- volatile conditions can change the attractiveness of posted curves or offers,
- arbitrage profitability depends on external price discrepancies.

Mitigations.

- deterministic execution primitives with explicit user protections (`minOut`, `deadline`),
- maker-controlled pricing surfaces (MAM curves) that are not trade-impact repriced,
- unified liquidity and fee rails that reduce fragmentation and encourage deeper participation over time.

11.2.5 MEV and Ordering Attacks

MEV cannot be eliminated in a public mempool environment. The protocol assumes:

- sandwich attempts on trade-like operations,
- backrunning of arbitrage opportunities,
- priority gas auctions and ordering manipulation.

Mitigations by surface.

- **MAM Curves:** reduce MEV vectors relying on trade-induced repricing; deterministic price at time t plus `minOut`/`deadline` protections.
- **AMM Auctions:** fee routing and deterministic accounting; takers can use slippage bounds; makers can adjust inventory and participation.
- **EqualIndex Flash Loans:** atomic execution reduces settlement risk; fees are deterministic and paid in-underlying.

These mitigations do not claim “MEV resistance” as an absolute. They aim to reduce the most harmful and extractive MEV patterns (especially slippage-based extraction) and make remaining competition more legible.

11.2.6 Accounting and Index Manipulation Risk

Any index-based distribution system must defend against:

- temporal gaming (entering right before fee events),
- wash activity intended solely to earn index rewards,
- multi-position strategies that attempt to double-count capital.

Mitigations.

- Fee Index earning weight normalized by same-asset debt to prevent borrow-deposit double counting,
- Active Credit Index time gating and weighting rules to reduce short-lived cycling dominance,
- eligibility constraints limiting ACI weight to defined active categories that align with fee-producing behavior,
- deterministic encumbrance preventing the same units of liquidity from satisfying conflicting states.

11.2.7 Module-Specific Blast Radius

Even with unified pools, risk must not be socialized by default. Modules can introduce localized hazards:

- execution modules can be exploited if mispriced or misaccounted,
- basket mechanisms can be abused if fee routing or vault accounting is incorrect,
- auction logic can be attacked via edge-case arithmetic or rounding.

Mitigations.

- strict encumbrance namespaces and accounting isolation (module balances are tracked distinctly),
- bounded permissions for module actions (only modules can mutate their encumbrance records),
- deterministic settlement rules with explicit invariants,
- phased rollout and progressive freezing of stable modules.

11.3 Invariants (System Safety Properties)

The protocol aims to preserve a set of invariants that can be tested and audited:

- **Conservation:** internal ledger balances plus encumbered balances reconcile to pool totals (modulo explicit fee flows).
- **Non-double-use:** an encumbered unit of liquidity cannot simultaneously satisfy withdrawal and module allocation.
- **Bounded deterministic credit:** self-secured debt in token T respects $D_{p,T} \leq \lambda_T \cdot \text{Deposit}_{p,T}$.
- **Index monotonicity:** I_T and A_T are non-decreasing and only increase via routed fee events.
- **Agreement determinism:** peer-to-peer agreement state transitions follow explicit rule sets (payment, grace, default, and optional call).
- **Vault correctness (EqualIndex):** minted supply is fully backed by underlying vault balances plus explicit accounting for fee pots.

11.4 Summary

Equalis reduces certain systemic DeFi risks by design (oracle-triggered liquidations and fragmented incentive islands) while accepting that other risks remain (smart contract risk, explicit credit default risk, and MEV). The mitigation strategy is to make risk legible, constrain blast radius through deterministic accounting and module isolation, and progressively remove discretionary control surfaces over time.

12 Governance and the Road to Immutability

Equalis is built with a constraint that many protocols acknowledge but few operationalize: the **walkaway test**. A system that claims to be trust-minimized should be able to continue operating correctly even if the team disappears and no one is left to “manage” it.

In practice, early-stage protocols need the ability to patch bugs, harden invariants, and respond to unexpected edge cases. Equalis therefore treats upgradeability as **temporary scaffolding**, not a permanent control surface. The end goal is a progressively immutable protocol where discretionary knobs are minimized, bounded, and ultimately removed wherever possible.

12.1 Phased Deployment Philosophy

Equalis is deployed in phases:

- **Phase 0 (Experimental):** rapid iteration, aggressive testing, and frequent fixes while invariants are validated.
- **Phase 1 (Stabilization):** reduced change frequency, formal audits, and focused hardening of core accounting and credit invariants.
- **Phase 2 (Freezing):** progressive immutability of mature surfaces; only bounded parameters remain adjustable.
- **Phase 3 (Infrastructure):** protocol behavior becomes predictable and “boring” in the best sense: rules do not change under users.

This approach acknowledges reality (software hardening takes time) while maintaining a clear destination (minimized trust assumptions).

12.2 Modular Architecture and Selective Freezing

Equalis is modular by design. Modules (credit primitives, auctions, index baskets, execution curves) are implemented as discrete components. This supports **selective freezing**:

- core accounting and pool invariants can be frozen first,
- higher-risk or more experimental modules can remain mutable longer,
- mature modules can be locked permanently once confidence is established.

Selective freezing prevents the system from remaining indefinitely upgradeable in the name of flexibility. Instead, flexibility is earned early, and removed deliberately over time.

12.3 Minimizing Control Surfaces

A protocol can be “governed” in ways that are functionally indistinguishable from administration. Equalis aims to avoid this by following two rules:

1. If a parameter can be abused, it should not exist.
2. If a parameter must exist, it should be **bounded** and **transparent**.

Examples of parameters that should be bounded if present include:

- fee rates and fee split proportions,
- per-token deterministic credit bounds (e.g., self-secured LTV),
- module-specific fee configuration where required for sustainability.

Fee-gated creation and action fees (implementation). The current implementation includes fee-gated pool creation, fee-gated index creation (when enabled), and per-action flat fees (borrow/repay/withdraw/flash/close) at the pool level. These should be treated as bounded parameters if retained in a long-term immutable configuration.

The protocol does not rely on governance to “actively manage” markets (e.g., rebalancing indices, changing compositions, or steering price). Where market behavior is needed, it is expressed through explicit offers (P2P credit), maker-set curves (MAM), or maker-set inventory and fee choices (auctions).

12.4 Upgrade Policy and Trust Assumptions

Early in the lifecycle, upgrades may be required to:

- fix correctness bugs,
- patch vulnerabilities,
- harden invariants discovered through adversarial use,
- optimize gas and reduce unnecessary complexity.

This is acknowledged explicitly: **early-stage safety depends on engineering and responsible upgrade practices.** The intended evolution is to reduce this trust requirement by freezing mature surfaces and limiting remaining mutability to bounded fee and configuration parameters.

12.5 Current Upgrade and Admin Surfaces (Implementation)

EIP-2535 Diamond architecture. The current codebase is deployed as an EIP-2535 **Diamond**: a single contract address that dispatches calls by function selector to **facets** via `delegatecall`. This provides modularity and reduces the need for state migrations, but it also introduces an explicit *upgrade surface* in early phases.

Facet upgrades (diamond cuts). Selector routing can be changed via `diamondCut` (adding, replacing, or removing selectors). In the current implementation, `diamondCut` is gated by the Diamond contract owner. Deployments that require upgrade delay can set the Diamond owner to a timelock contract (so upgrades are effectively timelocked), or otherwise constrain upgrades by operational policy.

Owner vs timelock roles. In addition to the Diamond owner, the implementation tracks a separate `timelock` address that is authorized alongside the owner for many parameter updates. This supports a workflow where routine configuration changes are executed by a delayed timelock, while ownership can later be transferred to (or permanently set as) the timelock to eliminate immediate-upgrade pathways.

Concrete bounded knobs (non-exhaustive). The current implementation exposes admin-controlled parameters such as:

- treasury and fee-routing split parameters (Treasury vs index rails),
- pool creation fees, index creation fees, and Position mint fees,
- per-action flat fees (borrow/repay/withdraw/flash/close) and fee bounds,
- bounded AUM fees (mutable within immutable min/max ranges),

- rolling delinquency and penalty thresholds,
- Equalis Direct rolling-offer validation bounds (payment cadence, APY bounds, penalty bps, minimum payments),
- derivative-module fee configuration (maker-share splits and flat/bps fee parameters),
- narrow pause switches for selected higher-risk surfaces (e.g., Atomic Desk).

These controls are intended as temporary scaffolding: they should be progressively bounded and, where possible, removed or frozen as the protocol approaches the walkaway end state.

12.6 Emergency Controls

If emergency controls exist during early phases, they should be treated as temporary and constrained. Any emergency mechanism should be:

- **narrow:** capable of preventing harm (e.g., pausing a specific module) without global discretionary power,
- **auditable:** clearly documented, event-emitting, and visible to users,
- **sunsettable:** designed to be removed or permanently disabled as surfaces are frozen.

The goal is not to promise that emergencies never happen. The goal is to ensure emergency response does not become permanent governance.

12.7 The End State

The long-term target is a protocol that is:

- **predictable:** rules and invariants do not change unexpectedly,
- **minimally discretionary:** few to no admin-controlled parameters,
- **hard to capture:** no central coordinator to pressure, and no privileged actor required for ordinary operation,
- **composable infrastructure:** stable enough for others to build upon without fearing rule changes.

In short: Equalis should become infrastructure that can pass the walkaway test, not a product that users are renting from an operator.

13 Implementation Overview and System Invariants

This section provides a high-level implementation overview of the Equalis system and enumerates core invariants that define correct behavior. The intent is not to provide a full specification of every function, but to describe the components that must exist for the architecture in this paper to be real: unified pools, Position NFTs, encumbrance, fee routing, indexes, and module settlement.

13.1 Core State Machines

13.1.1 Pool State (Per Token)

For each token T , the pool $\mathcal{P}_T$ maintains:

- **Totals:** total deposits, total encumbered balances, and any buffers required for fee routing.
- **Indexes:** global I_T (Fee Index) and A_T (Active Credit Index).
- **Parameters:** deterministic credit bounds (e.g., self-secured LTV λ_T), fee routing configuration, module eligibility flags, and action/maintenance fee settings.

Pool-level operations include:

- deposit/withdraw (ledger updates),
- self-secured borrow/repay (same-asset debt accounting),
- index updates via fee router (monotonic index increments),
- encumbrance allocate/release hooks (called by modules).

13.1.2 Position NFT State

A Position NFT stores the per-position accounting necessary to compute balances and claims:

- per-token ledger balances (free and encumbered),
- same-asset debt per token (self-secured credit),
- module encumbrance records (by namespace),
- peer-to-peer agreement references (as borrower or lender),
- per-token index snapshots (last seen I_T and A_T),
- accrued but unclaimed index amounts (optional caching).

The Position NFT is the unit of composability: a user's entire onchain balance sheet is represented as one transferable object (subject to solvency and module constraints).

13.1.3 Encumbrance Engine

Encumbrance is implemented as strict accounting that prevents double-use:

- modules request encumbrance allocations from the pool on behalf of a position,
- pools update the position's free/encumbered balances and record module-specific state,
- modules later release encumbrance to return balances to free liquidity.

A minimal encumbrance interface must support:

- allocate (p, T, M, x) : move x from free to encumbered under namespace M ,
- release (p, T, M, x) : move x from encumbered to free under namespace M ,
- query: expose encumbered totals and per-namespace commitments for verification and UI.

13.1.4 Fee Router and Index Updater

The fee router is the canonical entry point for distributing fees in token T . Given an incoming fee amount in T , it applies a configured split:

- direct participant payouts (maker shares, where applicable),
- protocol/treasury allocation,
- Fee Index allocation F_T^{FI} ,
- Active Credit Index allocation F_T^{ACI} .

Then it updates:

$$I_T \leftarrow I_T + \frac{F_T^{FI}}{W_T^{FI}}, \quad A_T \leftarrow A_T + \frac{F_T^{ACI}}{W_T^{ACI}},$$

with denominators computed from current earning weights and subject to non-zero distribution rules.

13.2 Module Settlement Surfaces (High-Level)

Equalis modules share a common pattern:

1. allocate inventory via encumbrance,
2. perform deterministic settlement (fills, agreements, mint/burn),
3. route fees into per-token fee routers,
4. release encumbrance when obligations end or inventory is withdrawn.

Modules discussed in this paper include:

- **Equalis Direct:** offer posting, fills, agreement lifecycle, default handling, optional callability.
- **AMM Auctions:** maker reserve allocation, taker swaps, maker fee distribution, index fee routing.
- **MAM Curves:** curve create/update lifecycle, deterministic time-based pricing, partial fills, fee splits.
- **EqualIndex:** mint/burn accounting, per-underlying fee pots, basket flash loans and per-underlying fee routing.

Additional implemented modules (current codebase). The implementation also includes tokenized **Options** and **Futures** series, an **Atomic Desk** and settlement escrow surface, and account-abstraction components (Position Agents, ERC-6551 TBAs, and ERC-6900 modules). These are compatible with the same pool/encumbrance/index architecture but are not fully specified in the core sections above.

13.3 Implementation Architecture: Diamond + Facets (Implementation)

The reference implementation is organized as an EIP-2535 Diamond. Conceptually, this paper describes “modules” as protocol surfaces with shared pool accounting and fee routing; in code, those surfaces are realized as facets and libraries that share a single storage layout.

Facet grouping (high level). The current codebase groups functionality into facets for: pool and position operations (deposit/withdraw/borrow/repay), Equalis Direct (offers, agreements, lifecycle), derivatives (tokenized options/futures), execution venues (AMM auctions and MAM curves), EqualIndex, administrative configuration, and read-only view/preview endpoints. Diamond loupe introspection enables tooling and indexers to enumerate selectors and facet addresses.

Upgrade surface. Facet routing may be changed by diamond cuts. This is an implementation choice; protocol-level invariants are intended to remain stable under upgrades, and upgradeability is treated as temporary scaffolding as described in the governance section.

13.4 Position Agents (ERC-6551 + ERC-6900 + ERC-8004) (Implementation)

In addition to the Position NFT as an account container, the current codebase optionally supports a **Position Agent** layer for automation and standardized discovery. This layer is *orthogonal* to core solvency: it does not introduce oracle-driven liquidation and it does not relax deterministic credit constraints.

ERC-6551 token-bound accounts (TBAs). For a given Position NFT, a deterministic token-bound account address can be computed from a canonical ERC-6551 registry and a configured account implementation. The Position owner can deploy this account on demand. The TBA can hold assets, sign messages, and execute transactions as a programmable “agent wallet” for the Position.

ERC-6900 modular smart contract account (MSCA). The TBA implementation follows an ERC-6900-style modular architecture: execution modules expose callable actions and validation modules define authorization rules. This allows Position behavior to be extended without changing core pool accounting.

Scoped session keys. A session-key validation module enables bounded delegation to an ephemeral key: policy can restrict validity windows, value limits, and which targets/selectors may be invoked. This supports bots and autonomous agents that can operate within explicit constraints rather than inheriting full Position control.

Skill modules (example). The current implementation includes an “AMM skill” execution module that allows a Position Agent to (i) create or cancel AMM auctions and (ii) roll accrued yield into principal, subject to policy bounds (duration, fee bps, reserve ranges, and optional pool allowlists). This is an automation surface; it does not modify the underlying solvency model.

ERC-8004 agent identity (optional discovery). For cross-application discovery and indexing, a Position Agent can be registered in the canonical ERC-8004 Identity Registry. Registration itself is performed externally (e.g., via the TBA executing against the registry), and the protocol records the mapping from Position tokenId $\rightarrow$ agentId after verifying that the ERC-8004 Identity NFT is owned by the Position’s TBA.

13.5 Admin and Timelock Parameter Machinery (Implementation)

The current implementation includes explicit configuration surfaces intended to be **bounded** and progressively frozen. Access control distinguishes between the Diamond owner and a separate configured **timelock** address that is authorized for many governance-minimized updates.

Representative configuration categories (non-exhaustive).

- **Fee routing:** treasury address; fee split shares between Treasury and index rails; managed-pool system-share routing.
- **Creation costs:** pool creation fee; index creation fee; Position mint fee (token and amount).
- **Pool configuration:** deposit/loan thresholds; deposit caps and user-count caps; maintenance rate bounds; flash-loan fee parameters; fixed-term configuration arrays.
- **Bounded mutability:** AUM fee adjustable within immutable per-pool min/max bounds; per-action flat fee overrides; pool deprecation flags for UI guidance.
- **Credit enforcement parameters:** rolling delinquency and penalty thresholds; minimum rolling payment parameters.
- **Direct rolling offer bounds:** payment cadence bounds; APY bounds; penalty bps; maximum upfront premium bounds.
- **Module fee parameters:** maker-share splits and fee configuration for execution/derivative modules.

These knobs should be treated as implementation scaffolding: they exist to enable safe iteration and operational hardening early, and should be reduced over time to pass the walkaway test.

13.6 System Invariants

The protocol is defined by invariants that must hold under all valid state transitions. These invariants are intended to be:

- **auditable** (derivable from onchain state),
- **testable** (expressible as property tests),
- **enforceable** (violations indicate a bug or exploit).

13.6.1 Accounting and Conservation

- **Ledger conservation (per token):** the sum of all position free balances plus the sum of all position encumbered balances reconciles to pool totals, modulo explicitly tracked fee buffers and protocol-owned balances.
- **No negative balances:** free and encumbered balances cannot underflow; debt cannot underflow.
- **Non-double-use:** a unit of liquidity marked encumbered under namespace M cannot simultaneously be counted as free liquidity or encumbered under a different conflicting namespace.

13.6.2 Deterministic Credit Safety

For self-secured credit in token T :

$$D_{p,T} \leq \lambda_T \cdot \text{Deposit}_{p,T} \quad \forall p,$$

where $\text{Deposit}_{p,T}$ is defined by pool accounting (e.g., free+encumbered deposit-side balance) and $D_{p,T}$ is same-asset debt.

13.6.3 Index Correctness

- **Monotonicity:** I_T and A_T are non-decreasing.
- **Fee-only increments:** indexes only increase due to routed fee events, not arbitrary writes.
- **Snapshot integrity:** position accrual depends on differences between current indexes and position snapshots; snapshot updates must be performed consistently on state-changing actions to prevent double-claims.

13.6.4 Agreement Determinism (P2P)

- **Valid transitions:** agreements may only move through valid lifecycle states (active $\rightarrow$ repaid, active $\rightarrow$ default, active $\rightarrow$ called if callable, etc.).
- **Default rules are explicit:** default is triggered by missed obligations beyond grace (or other explicit terms), not by external price events.
- **Callability is opt-in:** lender calls are only valid when the agreement was instantiated with callability enabled.

13.6.5 EqualIndex Correctness

- **Full backing:** index supply corresponds to underlying vault backing plus explicit fee pot accounting.
- **Deterministic redemption:** burns redeem pro-rata vault and pot shares, less explicitly defined fees.
- **Flash loan atomicity:** basket flash loans must return the full basket amounts plus per-underlying fees within the same transaction or revert.

13.7 Testing and Verification Approach (Overview)

A protocol with strict invariants should be tested accordingly:

- unit tests for every module state transition,
- invariant tests for conservation, non-double-use, and index monotonicity,
- fuzzing of arithmetic edges (rounding, overflow/underflow, boundary conditions),
- adversarial simulations (MEV ordering, partial fills, concurrent encumbrance operations),
- integration tests for multi-module compositions (e.g., P2P $\rightarrow$ swap $\rightarrow$ deposit loops, index mint/burn via Position accounting).

13.8 Summary

Implementation correctness in Equalis is defined by a small set of hard invariants: unified per-token pools, strict encumbrance accounting, deterministic credit bounds, monotonic index distribution, and explicit agreement enforcement. These properties are the foundation for the system-level claims in this paper: reduced dependence on liquidation markets, reduced liquidity fragmentation, and incentives that reward both passive and active participation through shared fee rails.

14 Conclusion

Equalis is an attempt to reframe onchain finance around deterministic rules, unified liquidity, and incentives that reward both passive and active participation without relying on liquidation markets as the default enforcement layer.

The core thesis is simple:

- **Deterministic solvency for the core:** self-secured credit can be high-LTV and 0% interest because collateral and debt are denominated in the same asset, removing the need for oracle-based liquidation to preserve solvency.
- **Explicit risk where risk is unavoidable:** cross-asset borrowing is handled through peer-to-peer agreements with legible terms, optional covenants (including opt-in callability), and deterministic default handling.
- **Unified liquidity by token:** a canonical base pool per token (with optional managed pools), with encumbrance-based allocation, prevents liquidity and incentives from fragmenting across products.
- **System-wide fee rails:** a dual index system routes value back to participants in a way that keeps passive liquidity rewards and active deployment rewards distinct and difficult to game.
- **Fee-producing execution surfaces:** AMM auctions, MAM curves, and EqualIndex basket primitives expand what capital can do while feeding the same per-token fee rails.

This architecture leads to a precise, defensible form of “perpetual leverage”: positions that are not subject to price-triggered forced unwinds by protocol rules. Continuation depends on explicit contractual obligations and voluntary unwind, not on maintaining a health factor against a price oracle.

Equalis does not claim to eliminate risk or guarantee returns. Smart contract risk remains a primary concern, especially in early stages. Peer-to-peer credit includes counterparty default risk. Market conditions affect the feasibility and profitability of strategies. The claim is narrower and stronger: **risk becomes legible**, enforcement becomes deterministic, and liquidity is not forced to fragment just to support a wider product surface.

The long-term goal is to pass the walkaway test. Equalis treats upgradeability as temporary scaffolding and proposes a path to progressively freeze mature surfaces, minimize discretionary knobs, and become predictable infrastructure that others can build on without fear of rule drift.

If DeFi is to become durable, it must move beyond systems that depend on reactive forced selling and fragile coordination assumptions. Equalis is one design exploration in that direction: socialize gains through unified fee rails, isolate risk through explicit agreements and module accounting, and let cryptographic rules, not intermediaries, determine outcomes.

A Glossary

Pool ($\mathcal{P}_T$) The canonical base pool for token T that holds unified accounting and fee rails (with optional managed pools routed back to it).

Position NFT An ERC-721 account container representing a complete balance sheet: deposits, debt, encumbrances, agreements, and index claims.

Diamond (EIP-2535) A single contract address that routes function selectors to facet contracts via `delegatecall`, enabling modular upgrades and a unified entrypoint.

Facet A contract that implements a subset of the Diamond’s callable selectors; facets share the Diamond’s storage layout under `delegatecall`.

Timelock A delayed execution authority used to gate administrative actions (and optionally upgrades) to reduce discretionary key risk.

Position Agent An optional automation/discovery layer that binds a Position NFT to a programmable account (TBA/MSCA) and optional identity registration.

TBA (ERC-6551) A token-bound account whose address is deterministically derived from an NFT and a registry, enabling the NFT to control a smart account.

MSCA (ERC-6900) A modular smart contract account supporting installable execution and validation modules (e.g., scoped session keys).

Session Key A delegated key with explicitly scoped permissions (time bounds, target-selector allowlists, and value limits) for constrained automation.

Encumbrance Internal allocation mechanism that marks balances as committed to a module without leaving the pool accounting domain.

Fee Router Per-token routing mechanism that splits collected fees between direct participants, Treasury, and indexes.

Fee Index (I_T) Monotonic per-token index distributing passive fees to eligible liquidity providers.

Active Credit Index (A_T) Monotonic per-token index distributing active rewards to deployed/eligible capital (borrowed, locked, or encumbered).

Equalis Direct Peer-to-peer credit system with onchain offer posting, tranche offers, and explicit agreement enforcement.

Term Tranche Offer Lender-posted offer that escrows principal and supports multiple fills until depleted.

Ratio Tranche Offer Lender- or borrower-posted offer that sets collateralization via a ratio and supports variable fill sizes.

MAM Curves Maker Auction Market curves implementing time-varying linear pricing (ascending/descending/flat) with cheap updates and order-like behavior.

AMM Auctions Solo and Community auction venues using encumbered inventory to provide execution and fee flow.

EqualIndex Deterministic, fully backed basket token primitive with per-underlying fee pots and basket flash loans.

B Index Math and Accrual (Formalization)

For token T , let:

- I_T be the Fee Index, A_T be the Active Credit Index.
- F_T^{FI} be fees routed to I_T , F_T^{ACI} be fees routed to A_T .
- $w_{p,T}^{FI}$ be position p Fee Index weight, $w_{p,T}^{ACI}$ be position p Active weight.

Define totals:

$$W_T^{FI} = \sum_p w_{p,T}^{FI}, \quad W_T^{ACI} = \sum_p w_{p,T}^{ACI}.$$

Index increments:

$$I_T \leftarrow I_T + \frac{F_T^{FI}}{W_T^{FI}}, \quad A_T \leftarrow A_T + \frac{F_T^{ACI}}{W_T^{ACI}}.$$

Position accrual between snapshots:

$$\Delta y_{p,T}^{FI} = w_{p,T}^{FI} \cdot (I_T - I_{p,T}^{\text{last}}), \quad \Delta y_{p,T}^{ACI} = w_{p,T}^{ACI} \cdot (A_T - A_{p,T}^{\text{last}}).$$

A conservative Fee Index weight:

$$w_{p,T}^{FI} = \max(\text{Deposit}_{p,T} - \text{Debt}_{p,T}, 0).$$

Active weight is protocol-defined over eligible categories with maturity constraints:

$$w_{p,T}^{ACI} = \text{EligibleMaturedActive}(p, T).$$

C Encumbrance Accounting Model

For each position p and token T , track free and encumbered balances:

$$B_{p,T}^{\text{free}}, \quad B_{p,T}^{\text{enc}}.$$

An encumbrance allocation of size x under module namespace M requires:

$$B_{p,T}^{\text{free}} \geq x,$$

and updates:

$$B_{p,T}^{\text{free}} \leftarrow B_{p,T}^{\text{free}} - x, \quad B_{p,T}^{\text{enc}} \leftarrow B_{p,T}^{\text{enc}} + x.$$

A release of size x under namespace M requires:

$$B_{p,T}^{\text{enc}}(M) \geq x,$$

and updates:

$$B_{p,T}^{\text{enc}} \leftarrow B_{p,T}^{\text{enc}} - x, \quad B_{p,T}^{\text{free}} \leftarrow B_{p,T}^{\text{free}} + x.$$

Non-double-use invariant. Encumbered balances are unavailable for withdrawal or conflicting encumbrances until released.

D EqualIndex Formalization

Let an index have underlyings $\mathcal{U} = \{T_1, \dots, T_k\}$ and per-unit quantities $\mathbf{q} = (q_1, \dots, q_k)$.

Mint. Mint x index units requires deposits:

$$\text{Deposit}(T_i) = x \cdot q_i \quad \forall i.$$

Burn. Let S be total index supply, V_i vault backing, P_i fee pot for T_i . Burn x returns:

$$\text{RedeemVault}(T_i) = \frac{x}{S} \cdot V_i, \quad \text{RedeemPot}(T_i) = \frac{x}{S} \cdot P_i,$$

minus burn fees.

Basket flash loan. Borrow x index units temporarily receives:

$$\text{FlashBorrow}(T_i) = x \cdot q_i,$$

and must return:

$$\text{FlashRepay}(T_i) = x \cdot q_i + \phi_i(x),$$

within the same transaction.

Fee split. Per-underlying fees are split between system fee routing and index fee pots.

E MAM Curve Mechanics (Formalization)

A curve has $(t_0, D, P_0, P_1, V_{\max})$ and remaining volume V_{rem} .

Price at time t :

$$P(t) = \begin{cases} P_0 & t \leq t_0, \\ P_1 & t \geq t_0 + D, \\ P_0 + (P_1 - P_0) \cdot \frac{t-t_0}{D} & \text{otherwise.} \end{cases}$$

A taker paying `amountIn` quote receives:

$$\text{baseOut} = \frac{\text{amountIn} \cdot 10^{18}}{P(t)},$$

subject to $0 < \text{baseOut} \leq V_{\text{rem}}$ and user protections (`maxQuote`, `minOut`, `deadline`).

F Protocol-Perpetual Leverage (Formal Definition)

A leveraged position is **protocol-perpetual** if the protocol contains no state transition rule that forces collateral sale solely as a function of external price movement. Position termination must be due to voluntary unwind or violation of explicit contractual obligations (e.g., missed payment beyond grace, or opt-in callability when agreed).